Universidad Técnica Federico Santa María
Departameto de Matemáticas

Transformers para la predicción de series de tiempo no equiespaciadas

Memoria para optar al título de

Ingeniero Civil Matemático
(Ingeniería Civil Matemática)

Presentada por

Gerald Méndez

Profesor guía: Carlos Valle
Profesor co-guía: Francisco Cuevas

Valparaíso, 2025

Índice general

1. Introducción **1**
1.1. Contexto y motivación 1
1.2. Problema de investigación 2
1.3. Hipótesis y preguntas de investigación 2
1.4. Objetivos 3
1.4.1. Objetivo general 3
1.4.2. Objetivos específicos 3
1.5. Enfoque y resumen de la propuesta 4
1.6. Alcances y limitaciones 5
1.7. Estructura de la memoria 5

2. Estado del arte **7**
2.1. Modelos clásicos para series de tiempo irregulares 7
2.1.1. Regularización e imputación sobre una malla equiespaciada 7
2.1.2. Modelos en tiempo continuo: CARMA y modelos de espacio de estados 8
2.1.3. Procesos gaussianos para series irregulares 8
2.2. Aprendizaje profundo para series de tiempo irregulares 9
2.2.1. Redes recurrentes con manejo explícito de tiempo y *missingness* . . 9
2.2.2. Modelos en tiempo continuo: Neural ODE, ODE-RNN y GRU–ODE–Bayes 9
2.2.3. Ecuaciones diferenciales controladas y Neural CDE 10
2.3. Transformers para series de tiempo 11
2.3.1. Transformers para series equiespaciadas 11
2.3.2. Positional encodings continuos y representaciones del tiempo 11
2.3.3. Transformers sobre observaciones irregulares y modelos híbridos . . 12
2.3.4. Agregación por *patches* y baselines simples para series irregulares . 13
2.4. Clasificación de series de tiempo irregulares 14
2.4.1. Métodos probabilísticos y kernels para series irregulares 14
2.4.2. Modelos de deep learning para clasificación irregular 14
2.4.3. Transformers para clasificación de series irregulares 15
2.5. Discusión y brechas de investigación 15

3. Series de tiempo y formulación del problema **17**
3.1. Conceptos básicos de series de tiempo 17
3.1.1. Definición y notación 17
3.1.2. Series equiespaciadas vs. no equiespaciadas 18
3.2. Datos considerados en la memoria 19
3.3. Problema de predicción en series de tiempo 19
3.3.1. Predicción a partir de una ventana de historia 19

3.3.2. Horizontes de predicción y offsets 20
3.4. Construcción de ejemplos supervisados 20
3.4.1. Ventanas deslizantes y tamaño de la historia 20
3.4.2. Definición de la entrada y la salida de cada ejemplo 21
3.5. Formulación como problema de aprendizaje supervisado 22
3.5.1. Función objetivo 22
3.5.2. Entrenamiento, validación y prueba 22
3.6. Resumen del capítulo 22

4. Transformers para series de tiempo no equiespaciadas 24
4.1. Recordatorio de la arquitectura Transformer 24
4.2. Transformers en series de tiempo 25
4.2.1. Ventajas 25
4.2.2. Desafíos con timestamps irregulares 25
4.3. Diseño general del modelo propuesto 26
4.4. Construcción de la secuencia con token target 26
4.5. Embeddings y encodings 27
4.5.1. Embedding de valores 27
4.5.2. Encoding temporal continuo 27
4.5.3. Embedding de flag historia/target 28
4.6. Encoder Transformer y máscara de atención 28
4.6.1. Máscara de padding y máscara causal opcional 29
4.7. Token target y cabeza de regresión 29
4.8. Manejo de historias y horizontes variables 30
4.8.1. Historias de longitud variable 30
4.8.2. Offsets de predicción múltiples 30
4.9. Inferencia y pronósticos multi-step 31
4.10. Resumen del capítulo 31

5. Modelo propuesto para series de tiempo no equiespaciadas 32
5.1. Objetivos de diseño 33
5.2. Representación de entradas y construcción de ejemplos 33
5.2.1. Separación entre valores y timestamps 33
5.2.2. Ventanas de historia y targets en TimeSeriesDataset 34
5.2.3. Construcción de secuencias con token objetivo 35
5.3. Arquitectura del TimeSeriesTransformer no equiespaciado 35
5.3.1. Embedding de valores continuos 36
5.3.2. Encoding temporal continuo 36
5.3.3. Embedding de flag historia/target 37
5.3.4. Combinación de embeddings y paso por el encoder 38
5.3.5. Cabeza de regresión en el token objetivo 39
5.4. Manejo de historias variables y offsets de predicción 39
5.5. Función de pérdidas, métricas y bucle de entrenamiento 40
5.5.1. Función de pérdida 40
5.5.2. Métricas de evaluación 40
5.5.3. Optimización y scheduler 40
5.6. Inferencia y pronósticos multi-step 41
5.7. Comentarios sobre complejidad y limitaciones 41

6. Experimentos **43**
6.1. Objetivos experimentales . . . 43
6.2. Escenario experimental . . . 44
6.2.1. Variables, partición temporal y preprocesamiento . . . 44
6.3. Modelos comparados . . . 45
6.3.1. Arquitecturas propuestas . . . 45
6.3.2. Baselines del estado del arte . . . 45
6.3.3. Baselines simples . . . 45
6.3.4. Variantes de ablación . . . 46
6.4. Configuración de entrenamiento . . . 46
6.5. Estudio de Ablación Autoregresivo (Fine-Tuning) . . . 47
6.6. Métricas de evaluación . . . 47
6.7. Análisis estadístico . . . 48
6.8. Costo computacional . . . 48
6.9. Reproducibilidad . . . 48
6.10. Resumen metodológico . . . 49

7. Resultados y discusión **50**
7.1. Resultados generales del benchmark . . . 50
7.2. Comparación contra baselines . . . 50
7.2.1. Modelo propuesto versus STraTS . . . 50
7.2.2. Modelo propuesto versus baselines simples . . . 51
7.2.3. Consistencia por dataset . . . 51
7.3. Estudios de ablación . . . 51
7.4. Análisis de costo computacional . . . 52
7.5. Significancia estadística . . . 52
7.5.1. Test de Wilcoxon de rango con signo . . . 52
7.5.2. Intervalos de confianza bootstrap . . . 53
7.6. Interpretación general de los resultados . . . 53
7.7. Discusión sobre MAPE y estabilidad de las métricas . . . 53
7.8. Síntesis del capítulo . . . 54

8. Conclusiones y trabajo futuro **55**
8.1. Conclusiones principales . . . 55
8.2. Solidez de la evidencia experimental . . . 56
8.3. Limitaciones . . . 56
8.4. Trabajo futuro . . . 57
8.5. Cierre . . . 57

A. Fundamentos de redes neuronales y optimización **60**
A.1. Introducción . . . 60
A.2. Neurona artificial . . . 60
A.2.1. Modelo básico . . . 60
A.2.2. Interpretación geométrica . . . 61
A.3. Redes neuronales feedforward . . . 62
A.3.1. Capas densas y notación matricial . . . 62
A.3.2. Composición de capas . . . 62
A.4. Función de pérdida . . . 63
A.4.1. Pérdida para problemas de regresión . . . 63

A.4.2. Regularización en la función de pérdida 64
A.5. Descenso de gradiente y retropropagación 64
A.5.1. Descenso de gradiente 64
A.5.2. Retropropagación (*backpropagation*) 64
A.6. Funciones de activación 65
A.6.1. Requisitos básicos 65
A.6.2. Sigmoide y tangente hiperbólica 65
A.6.3. ReLU y variantes 66
A.7. Problemas en el entrenamiento de redes profundas 66
A.7.1. Gradientes que se desvanecen y explotan 66
A.7.2. Regularización y generalización 67
A.8. Optimizadores avanzados: Adam 67
A.8.1. Motivación 67
A.8.2. Algoritmo Adam 67
A.9. Relación con el modelo Transformer de la memoria 68
A.10. Resumen del anexo 69

Índice de figuras

A.1. Ejemplo bidimensional de un hiperplano que separa dos clases linealmente separables: las cruces negras pertenecen a una clase y los círculos rojos a la otra. 61
A.2. Esquema de una red neuronal *feedforward* con dos capas ocultas: los vectores de entrada $x_1, \ldots, x_{\text{in}}$ se transforman mediante las matrices de pesos W_1 y W_2 en representaciones internas, para finalmente producir las salidas $\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_{\text{out}}$ 63

Índice de cuadros

7.1. Métricas de test promediadas sobre 17 datasets reales (media ± std). Resultados con 3 semillas por combinación (dataset, modelo). Valores menores son mejores. 50
7.2. Número de datasets donde cada modelo obtiene el menor MSE sobre test. . 51
7.3. Resultados de ablación (MSE sobre test, media ± std sobre 17 datasets, 3 semillas). 51
7.4. Costo computacional promedio por modelo sobre todas las corridas del benchmark. 52
7.5. Tests de Wilcoxon emparejados por dataset (17 pares, métricas sobre test). 52

Capítulo 1

Introducción

1.1. Contexto y motivación

La predicción de series de tiempo es un problema central en múltiples áreas de la ingeniería y las ciencias aplicadas, tales como la minería, la energía, las telecomunicaciones, la meteorología o la salud. En estos dominios es habitual contar con grandes volúmenes de datos temporales recolectados por sensores, sistemas de monitoreo o plataformas transaccionales. Poder anticipar el comportamiento futuro de estas series permite, entre otras cosas, planificar operaciones, detectar anomalías, optimizar recursos y reducir costos asociados a fallas o ineficiencias.

Tradicionalmente, la modelación de series de tiempo se ha abordado mediante modelos estadísticos como ARIMA, modelos autorregresivos vectoriales (VAR) o modelos de suavizamiento exponencial. Más recientemente, el desarrollo de técnicas de *aprendizaje profundo* ha dado lugar a arquitecturas como las redes recurrentes (RNN), las redes LSTM y GRU, así como modelos convolucionales específicamente adaptados a datos temporales. Sin embargo, en los últimos años ha emergido con fuerza una familia de modelos basada en el mecanismo de atención, conocida como *Transformers*, que ha demostrado un rendimiento sobresaliente en tareas de procesamiento de lenguaje natural, visión computacional y, en menor medida, en series de tiempo.

La mayoría de las aplicaciones exitosas de Transformers a series de tiempo asume implícita o explícitamente que las observaciones están **equiespaciadas** en el tiempo; es decir, que el intervalo entre mediciones es constante. Esta suposición simplifica la definición de posiciones en la secuencia y permite reutilizar sin cambios el *positional encoding* sinusoidal clásico. No obstante, en muchos sistemas reales los datos no se registran con una frecuencia perfectamente regular: pueden existir lapsos de tiempo sin mediciones, lecturas adicionales cuando se detectan eventos relevantes, fallas en los sensores o cambios en la tasa de muestreo. En estas situaciones se habla de **series de tiempo no equiespaciadas**.

Trabajar con series no equiespaciadas introduce desafíos adicionales. Una práctica habitual consiste en *regularizar* la serie mediante técnicas como re-muestreo (por ejemplo, cada 5 minutos) e imputación de valores faltantes. Si bien esto permite reutilizar modelos diseñados para series equiespaciadas, también introduce sesgos y pérdida de información temporal, especialmente cuando la irregularidad es alta o cuando la dinámica del sistema depende críticamente de la distribución real de los tiempos de observación.

En este contexto, surge la motivación de investigar si es posible adaptar la arquitectura Transformer para trabajar de manera más directa con timestamps continuos, evitando imponer una grilla temporal artificial y aprovechando explícitamente la información sobre la separación temporal entre mediciones.

1.2. Problema de investigación

El problema que se aborda en esta memoria puede resumirse de la siguiente forma:

> *Desarrollar una estrategia para utilizar modelos tipo Transformer para la predicción de series de tiempo no equiespaciadas, incorporando los timestamps de forma explícita mediante un encoding temporal continuo, comparar su desempeño con enfoques basales que asumen series equiespaciadas y con modelos del estado del arte diseñados para manejar series no equiespaciadas.*

En particular, se busca responder si un Transformer diseñado para operar con tiempos reales (por ejemplo, segundos desde un origen de referencia), y no sólo con índices enteros de posición, es capaz de:

- Capturar adecuadamente la dinámica de series de tiempo donde los intervalos entre mediciones son variables.
- Mantener o mejorar el desempeño predictivo respecto de modelos entrenados sobre versiones re-muestreadas o imputadas de la misma serie.
- Ofrecer una formulación lo suficientemente flexible como para manejar ventanas de historia de longitud variable y distintos horizontes de predicción, sin necesidad de rediseñar la arquitectura.

Este problema combina aspectos conceptuales—cómo representar el tiempo de forma continua en un Transformer—con aspectos prácticos de implementación y experimentación—cómo construir datasets, definir ventanas temporales y diseñar experimentos reproducibles para comparar distintos enfoques.

1.3. Hipótesis y preguntas de investigación

A partir del problema descrito, se plantea la siguiente hipótesis central:

> **Hipótesis:** *Un modelo Transformer que incorpore un encoding temporal continuo basado en los timestamps reales de las observaciones puede modelar series de tiempo no equiespaciadas de manera competitiva o superior a enfoques que requieren re-muestreo e imputación sobre una grilla temporal fija.*

De esta hipótesis se desprenden varias preguntas específicas:

- ¿Qué tipo de encoding temporal continuo resulta adecuado para representar los timestamps en el espacio latente del modelo? ¿Un encoding sinusoidal continuo es suficiente o se requieren componentes aprendibles adicionales?

- ¿Cómo afecta la irregularidad en los tiempos de muestreo al desempeño de un Transformer con encoding temporal continuo, en comparación con un Transformer estándar entrenado sobre datos re-muestreados?
- ¿Es posible diseñar un esquema de construcción de ventanas de historia y horizontes de predicción que permita al modelo entrenarse con longitudes de contexto variables y distintos offsets de predicción?
- ¿Qué impacto tienen las decisiones de entrenamiento (función de pérdida, optimizador, normalización de datos) en la estabilidad y desempeño del modelo en este escenario no equiespaciado?

Responder estas preguntas requiere tanto de una formulación conceptual clara de la arquitectura propuesta como de una campaña experimental controlada, que permita aislar el efecto de la representación temporal respecto de otros factores.

1.4. Objetivos

1.4.1. Objetivo general

El objetivo general de esta memoria es:

> *Diseñar, implementar y evaluar un modelo tipo Transformer para la predicción de series de tiempo no equiespaciadas, incorporando explícitamente los timestamps mediante un encoding temporal continuo, y analizar su desempeño y limitaciones en comparación con enfoques basales.*

1.4.2. Objetivos específicos

Para alcanzar este objetivo general, se plantean los siguientes objetivos específicos:

1. **Revisar el estado del arte** en modelos de series de tiempo, con énfasis en arquitecturas de aprendizaje profundo (RNN, CNN, Transformers) y en propuestas que abordan explícitamente el problema de tiempos de muestreo irregulares.
2. **Formalizar el problema** de predicción de series de tiempo no equiespaciadas, definiendo la notación, las variables involucradas y los escenarios de predicción a considerar (un paso adelante, múltiples pasos, distintos horizontes, etc.).
3. **Diseñar una arquitectura Transformer** capaz de:
   - Proyectar features continuas multivariadas a un espacio latente común.
   - Incorporar un encoding temporal continuo basado en timestamps reales y una escala de tiempo configurable.
   - Diferenciar explícitamente entre tokens de historia y tokens objetivo mediante embeddings adicionales.
4. **Implementar un pipeline completo** de entrenamiento e inferencia que incluya:
   - Construcción de ventanas de historia y targets para series no equiespaciadas.

- Entrenamiento del modelo con historias de longitud fija o variable, y con uno o varios offsets de predicción[1] posibles, lo que posibilita estudiar el efecto del horizonte de predicción sobre el desempeño.
- Módulos de escalado, funciones de pérdida, métricas y bucles de entrenamiento reproducibles.

5. **Diseñar y ejecutar experimentos** sobre datos sintéticos y datos reales que permitan comparar:

   - El modelo Transformer propuesto con encoding temporal continuo.
   - Baselines tradicionales (por ejemplo, modelos entrenados sobre series re-muestreadas equiespaciadas).

   utilizando métricas de error de regresión (MSE, RMSE, MAE, MAPE) y análisis cualitativo de curvas de predicción.

6. **Analizar los resultados** para evaluar la viabilidad del enfoque, identificar sus ventajas y limitaciones, y proponer posibles líneas de trabajo futuro.

1.5. Enfoque y resumen de la propuesta

Para abordar los objetivos anteriores, en esta memoria se desarrolla una arquitectura basada en el modelo Transformer adaptada específicamente a series de tiempo no equiespaciadas. De forma muy resumida, la propuesta se estructura en los siguientes componentes:

- Un **módulo de embedding de valores** que proyecta las variables de entrada (por ejemplo, mediciones de sensores) desde un espacio de dimensión d_{in} a un espacio latente de dimensión d_{model}, común a todo el modelo.
- Un **encoding temporal continuo** que recibe directamente los timestamps numéricos y construye vectores de dimensión d_{model} para cada observación, utilizando una normalización en base a una escala de tiempo configurable y un esquema sinusoidal (o, alternativamente, un MLP) inspirado en el positional encoding clásico.
- Un **embedding de flag de target** que diferencia explícitamente entre tokens de historia y tokens objetivo, permitiendo introducir un *token target* en la secuencia con su timestamp objetivo, pero sin conocer aún su valor verdadero.
- Un **encoder Transformer** compuesto por múltiples bloques de atención multicabeza y capas feed-forward, que opera sobre la secuencia completa (historia + token target), respetando opcionalmente una máscara causal.
- Una **cabeza de regresión** que toma la representación latente del token target a la salida del encoder y la proyecta al espacio de salida, entregando la predicción para el valor de interés en el timestamp objetivo.

[1] En este contexto, un *offset de predicción* es la distancia entre el último punto de la historia usada como entrada y el instante futuro que se desea predecir. Por ejemplo, un offset de 1 puede corresponder al siguiente punto de la serie, mientras que un offset de 20 puede representar predecir veinte pasos más adelante.

Sobre esta arquitectura se construye un conjunto de herramientas de datos y entrenamiento que permiten:

- Generar ventanas de historia y targets desde una serie temporal ordenada, respetando la naturaleza no equiespaciada de los timestamps.
- Entrenar el modelo con historias de longitud fija o variable, y con uno o varios offsets de predicción posibles, lo que posibilita estudiar el efecto del horizonte de predicción sobre el desempeño.
- Evaluar el modelo mediante un conjunto de métricas de regresión estándar y analizar su comportamiento tanto en datos sintéticos como en datos reales.

Los fundamentos matemáticos de redes neuronales, funciones de activación, algoritmos de optimización (incluido Adam) y aspectos generales de entrenamiento de modelos de aprendizaje profundo se desarrollan en un anexo independiente, con el fin de mantener el cuerpo principal de la memoria enfocado en la propuesta específica y sus resultados, sin perder rigurosidad en la presentación de los conceptos básicos.

1.6. Alcances y limitaciones

Este trabajo se concentra en el problema de **predicción puntual** de series de tiempo, es decir, en entregar un valor esperado para la variable de interés en uno o varios tiempos futuros. No se abordan explícitamente modelos probabilísticos completos (por ejemplo, distribución de probabilidad sobre futuros posibles) ni técnicas de *forecasting* probabilístico como cuantiles o intervalos de confianza, aunque se mencionan brevemente como posibles extensiones.

Asimismo, la evaluación se realiza sobre un conjunto acotado de datasets, que incluyen datos sintéticos controlados y al menos un caso de datos reales representativos de un sistema físico o de operación. El objetivo no es presentar un modelo universalmente óptimo para todas las series de tiempo no equiespaciadas, sino estudiar en detalle la viabilidad y el comportamiento de la arquitectura propuesta en escenarios concretos.

Por último, si bien la implementación se realiza sobre la biblioteca PyTorch y se incluye un pipeline completo de entrenamiento e inferencia, aspectos de despliegue en producción, integración con sistemas de monitoreo en tiempo real o escalamiento distribuido quedan fuera del alcance de esta memoria y se mencionan únicamente como posibles líneas de trabajo futuro.

1.7. Estructura de la memoria

El resto de este documento se organiza de la siguiente manera:

- En el **Capítulo 2** se presenta el estado del arte en modelos de series de tiempo, revisando tanto enfoques clásicos como arquitecturas de aprendizaje profundo, con énfasis en Transformers y en técnicas para manejar series no equiespaciadas.
- En el **Capítulo 3** se formaliza el problema de predicción de series de tiempo no equiespaciadas, se introduce la notación utilizada y se describen los datasets empleados en los experimentos.

- En el **Capítulo 4** se revisan los componentes principales de la arquitectura Transformer aplicada a series de tiempo equiespaciadas, destacando el rol del positional encoding y las particularidades del uso de atención en este contexto.
- En el **Capítulo 5** se detalla la propuesta de modelo Transformer para series no equiespaciadas, incluyendo el diseño del encoding temporal continuo, la construcción de secuencias con token target y la integración con el encoder Transformer y la cabeza de salida.
- En el **Capítulo 6** se describe la metodología experimental, los escenarios considerados y los procedimientos de entrenamiento y evaluación utilizados.
- En el **Capítulo 7** se presentan y analizan los resultados obtenidos, comparando el modelo propuesto con distintos baselines y discutiendo las implicancias de los hallazgos.
- Finalmente, en el **Capítulo 8** se resumen las conclusiones principales, se evalúa el cumplimiento de los objetivos planteados y se proponen líneas de trabajo futuro.

Los anexos incluyen una exposición más detallada de los fundamentos de redes neuronales y algoritmos de optimización utilizados, así como información técnica adicional sobre la implementación y resultados complementarios de los experimentos.

Capítulo 2

Estado del arte

En este capítulo se revisan los principales enfoques para modelar series de tiempo con marcas de tiempo irregulares, tanto para tareas de *forecasting* como de clasificación. El objetivo es situar la propuesta de esta memoria dentro del contexto de la literatura reciente, destacando las ventajas y limitaciones de cada familia de modelos, y las brechas que motivan el modelo basado en *Transformers* descrito en los capítulos siguientes.

2.1. Modelos clásicos para series de tiempo irregulares

2.1.1. Regularización e imputación sobre una malla equiespaciada

Una estrategia ampliamente utilizada para tratar series no equiespaciadas consiste en transformar el problema a un escenario equiespaciado. Típicamente se elige una malla temporal fija (por ejemplo, cada 5 o 15 minutos) y se aplica alguna combinación de:

- **Re-muestreo** (*resampling*) de las observaciones al grid objetivo.
- **Imputación** de valores faltantes mediante métodos simples (cero, promedio, *forward/backward fill*) o técnicas más avanzadas (interpolación, imputación múltiple, *splines*, *kalman smoothing*, etc.).

Sobre la serie regularizada se aplican luego modelos estándar como ARIMA, modelos de suavizamiento exponencial, redes recurrentes o *Transformers* para datos equiespaciados. Esta aproximación es sencilla de implementar y permite reutilizar modelos consolidados; sin embargo, presenta limitaciones relevantes:

- La elección del paso de malla introduce un **compromiso** entre resolución temporal y costo computacional.
- La imputación puede introducir **sesgos** sistemáticos cuando la irregularidad está asociada a la dinámica del sistema (por ejemplo, muestreo más denso durante eventos anómalos).
- Se pierde la posibilidad de formular horizontes de predicción directamente en tiempo continuo (por ejemplo, “15 minutos”, “1 hora”) de manera consistente con los *timestamps* originales.

Estas limitaciones motivan el desarrollo de modelos que trabajen directamente con tiempos continuos y sean capaces de manejar observaciones irregulares sin necesidad de re-muestreo agresivo.

2.1.2. Modelos en tiempo continuo: CARMA y modelos de espacio de estados

En el ámbito estadístico, una primera familia de modelos que trata de manera explícita el tiempo continuo son los procesos **CARMA** (*Continuous-time AutoRegressive Moving Average*) Brockwell, 2001a, 2001b. De forma análoga a los modelos ARMA discretos, los CARMA describen la evolución de un proceso estocástico mediante ecuaciones diferenciales lineales con ruido gaussiano, lo que permite modelar datos observados a tiempos irregulares sin re-muestreo explícito. La representación en espacio de estados facilita la inferencia mediante filtros y suavizadores de Kalman en tiempo continuo, y la evaluación de la verosimilitud para observaciones registradas en instantes arbitrarios.

Los modelos en espacio de estados continuos se han extendido a escenarios más generales (por ejemplo, procesos de difusión o modelos con parámetros dependientes del tiempo), pero siguen siendo mayoritariamente **lineales y gaussianos**. Esto limita su capacidad para capturar dinámicas altamente no lineales, frecuentes en aplicaciones industriales o biomédicas. Además, la extensibilidad a series multivariadas de alta dimensión suele implicar matrices de gran tamaño y costos de cómputo elevados.

2.1.3. Procesos gaussianos para series irregulares

Los **procesos gaussianos** (GP) constituyen otra herramienta probabilística natural para modelar series de tiempo irregulares, ya que la función de covarianza se evalúa directamente sobre los *timestamps* reales Williams y Rasmussen, 2006. Un GP define, para cualquier conjunto finito de instantes $\{t_1, \ldots, t_T\}$, una distribución multivariada gaussiana

$$(y(t_1), \ldots, y(t_T)) \sim \mathcal{N}(m, K),$$

donde $K_{ij} = k(t_i, t_j)$ está determinada por un *kernel* que codifica supuestos de suavidad, periodicidad o escala temporal.

Esta formulación tiene varias ventajas:

- Maneja **directamente** muestreo irregular: basta con evaluar el kernel en los tiempos observados.
- Proporciona **predicciones probabilísticas** (medias e intervalos de confianza) para cualquier tiempo futuro.
- Permite incorporar conocimiento de dominio mediante el diseño de kernels específicos (por ejemplo, kernels periódicos o *quasi-periodic*).

Sin embargo, la complejidad cúbica $O(T^3)$ en el número de observaciones limita el uso de GP exactos en series largas. Aunque existen aproximaciones escalables basadas en *inducing points*, estructuras de kernel especiales o submuestreo, el uso de GP puros en escenarios de alta frecuencia y larga duración sigue siendo desafiante, y la extensión a múltiples variables y tareas complejas (clasificación secuencial, detección de eventos) requiere diseños cuidadosos.

2.2. Aprendizaje profundo para series de tiempo irregulares

En la última década, los modelos de aprendizaje profundo han ganado protagonismo en el análisis de series de tiempo, tanto regulares como irregulares. A continuación se revisan tres líneas principales: (i) redes recurrentes con conocimiento explícito del tiempo, (ii) modelos en tiempo continuo basados en ecuaciones diferenciales, y (iii) modelos híbridos que combinan ideas de ambos mundos.

2.2.1. Redes recurrentes con manejo explícito de tiempo y *missingness*

Las **redes recurrentes** (RNN), incluyendo variantes como LSTM y GRU Cho et al., 2014; Chung et al., 2014; Gers et al., 2000; Hochreiter y Schmidhuber, 1997, son naturalmente adecuadas para procesar secuencias de longitud variable. Sin embargo, en su versión estándar asumen un paso de tiempo implícito (uno por paso de la RNN) y no modelan explícitamente los intervalos de tiempo entre observaciones. Para series irregulares, esto se ha abordado introduciendo *features* temporales y mecanismos de decaimiento aprendibles.

Un trabajo influyente en esta dirección es *GRU-D* Che et al., 2018, que extiende la GRU estándar incorporando dos ideas clave: (i) un vector de **máscaras** que indica qué variables están observadas o faltantes en cada instante, y (ii) un conjunto de **intervalos de tiempo** desde la última observación de cada variable, que se usan para definir factores de decaimiento aprendibles sobre las entradas y los estados ocultos. De este modo, el modelo puede aprender a "olvidar" información antigua y a imputar valores faltantes hacia medias empíricas de forma dependiente del contexto, mejorando el desempeño en tareas como clasificación de pacientes a partir de registros clínicos irregulares.

Otros trabajos relacionados incluyen:

- Modelos que concatenan directamente el *timestamp* o el intervalo de tiempo como entrada adicional a la RNN.
- Variantes de LSTM con puertas de olvido que dependen de funciones del intervalo de tiempo (por ejemplo, decaimientos exponenciales o polinomiales).
- Arquitecturas que combinan imputación explícita con RNN estándar, usando redes auxiliares para predecir valores faltantes antes de alimentarlos a la red recurrente.

En general, estos enfoques muestran que incorporar explícitamente el tiempo transcurrido entre observaciones y la estructura de datos faltantes mejora de manera consistente el rendimiento en *forecasting* y clasificación, especialmente en dominios como salud, donde la irregularidad y *missingness* son altamente informativas.

2.2.2. Modelos en tiempo continuo: Neural ODE, ODE-RNN y GRU–ODE–Bayes

Un segundo grupo de trabajos adopta una perspectiva de **tiempo continuo**, en la cual la dinámica latente de la serie se modela mediante ecuaciones diferenciales. En lugar de

actualizar el estado oculto en pasos discretos fijos, estos modelos integran una EDO entre observaciones, lo que permite manejar de manera natural tiempos de muestreo irregulares.

Entre las propuestas más destacadas se encuentran:

- **ODE-RNN** y **Latent ODE**: el estado latente evoluciona según una EDO parametrizada por una red neuronal (*Neural ODE*) entre observaciones, y se actualiza mediante saltos discretos al recibir nuevas mediciones. El entrenamiento se realiza mediante el método del *adjoint* continuo, permitiendo backpropagation eficiente a través del solucionador de EDO Chen et al., 2018; Yulia et al., 2019.

- **GRU–ODE–Bayes**: combina una dinámica continua basada en GRU–ODE con un tratamiento bayesiano de las observaciones, separando conceptualmente el proceso generativo continuo y el mecanismo de muestreo/observación. Esto permite manejar ruidos de medición y frecuencia de muestreo variable de manera coherente De Brouwer et al., 2019.

Estos modelos capturan con elegancia la naturaleza continua del tiempo y permiten evaluar el estado latente en cualquier instante, lo cual es atractivo para predicción multi-step y para fusionar series con distintos calendarios de muestreo. No obstante, la necesidad de resolver una EDO (o un conjunto de ellas) en cada paso introduce un costo computacional significativo, y la estabilidad numérica puede depender fuertemente de la elección del solucionador y de la parametrización de la dinámica.

2.2.3. Ecuaciones diferenciales controladas y Neural CDE

Los modelos basados en **ecuaciones diferenciales controladas** (CDE) generalizan la idea anterior modelando la trayectoria latente z_t como solución de una ecuación

$$\mathrm{d}z_t = f_\theta(z_t)\,\mathrm{d}X_t,$$

donde X_t es una versión suavizada o interpolada de la serie de entrada, considerada como un *camino control* en tiempo continuo. Las **Neural CDE** parametrizan f_θ mediante una red neuronal y resuelven la ecuación con un solucionador numérico, de forma análoga a las Neural ODE, pero interpretando las entradas como controles que impulsan la dinámica Kidger et al., 2020.

Esta formulación tiene varias ventajas para series irregulares:

- La interpolación continua de las observaciones permite manejar **cualquier calendario de muestreo**, siempre que se defina una manera razonable de rellenar los huecos entre puntos (por ejemplo, interpolación lineal o splines).

- El modelo es inherentemente **invariante a reparametrizaciones** del tiempo si se utilizan interpolaciones apropiadas, lo que lo hace robusto a cambios de resolución temporal.

- Se puede demostrar que las Neural CDE son capaces de aproximar una clase amplia de funcionales sobre caminos, lo que las hace apropiadas para tareas tanto de *forecasting* como de clasificación.

De nuevo, el costo computacional es mayor que en RNN estándar, y la complejidad de implementación (interpoladores, solucionadores, control de error) puede dificultar su adopción en entornos industriales. Además, la incorporación de mecanismos de atención similares a los de un *Transformer* no es directa en este marco.

2.3. Transformers para series de tiempo

Los *Transformers* han demostrado un rendimiento sobresaliente en procesamiento de lenguaje natural, visión por computador y, más recientemente, en forecasting de series de tiempo equiespaciadas Vaswani et al., 2017. En esta sección se revisan primero las extensiones para series regulares y, luego, las propuestas que abordan de manera explícita el muestreo irregular o el tiempo continuo.

2.3.1. Transformers para series equiespaciadas

El uso de *Transformers* en series de tiempo regulares se ha disparado en los últimos años. Numerosos trabajos han propuesto arquitecturas y componentes especializados para mejorar el rendimiento y la eficiencia computacional en pronósticos de largo horizonte, típicamente bajo el supuesto de observaciones equiespaciadas:

- **Informer**: introduce una atención *prob-sparse* para reducir la complejidad cuadrática a casi lineal en la longitud de la secuencia, manteniendo el foco en las contribuciones de atención más relevantes. Propone además un esquema de codificación temporal híbrida (posicional + *time-of-day*, *day-of-week*, etc.) y demuestra mejoras significativas en benchmarks de *forecasting* multivariante H. Zhou et al., 2021.

- **Autoformer**: replantea la auto-atención como un mecanismo de auto-correlación sobre descomposiciones tendencia–estacionalidad, proponiendo un bloque de *series decomposition* que separa componentes de baja frecuencia y patrones periódicos. Esta descomposición se integra en las capas del *Transformer*, mejorando la capacidad de modelar estructuras de largo plazo Wu et al., 2021.

- **FEDformer**: extiende la idea de Autoformer incorporando descomposición en dominios de frecuencia y empleando atención sobre componentes seleccionados en el espacio espectral, lo que reduce la carga computacional manteniendo la expresividad para series largas T. Zhou et al., 2022.

Si bien estas arquitecturas logran resultados de vanguardia en benchmarks populares, comparten supuestos clave:

- Las observaciones están en una malla **equiespaciada**; el índice de tiempo se puede tratar como un entero $1, \ldots, T$.

- El *positional encoding* se define típicamente sobre estos índices discretos, a veces enriquecido con *features* calendáricas (hora del día, día de la semana, etc.), pero no sobre *timestamps* reales.

Cuando se aplican a series irregulares, estos modelos suelen requerir un preprocesamiento de re-muestreo e imputación como el descrito en la Sección 2.1, con las limitaciones asociadas.

2.3.2. Positional encodings continuos y representaciones del tiempo

Una línea de trabajo más reciente se enfoca en **representaciones continuas del tiempo** para integrarlas en arquitecturas tipo *Transformer*. Entre las propuestas relevantes se encuentran:

- **Time2Vec**: introduce una representación aprendible del tiempo que combina una componente lineal y varias componentes periódicas parametrizadas, generando un vector $v(t)$ que captura tanto tendencias como periodicidades en el tiempo. Time2Vec puede concatenarse a las entradas de un *Transformer* o de otros modelos Kazemi et al., 2019.

- Trabajos que estudian la relación entre **positional encodings discretos y continuos**, mostrando que los codificadores sinusoidales clásicos pueden interpretarse como evaluaciones discretas de funciones definidas sobre tiempo continuo, y explorando extensiones que permiten evaluar el encoding en tiempos reales intermedios. Esto abre la puerta a reemplazar el índice entero por un *timestamp* real normalizado.

Estas ideas son particularmente relevantes para series irregulares: si el *positional encoding* puede evaluarse en cualquier tiempo, es posible definir un **encoding temporal continuo** que reciba directamente los *timestamps* reales, preservando la estructura del *Transformer* original. Esta es precisamente la filosofía que se adopta en el modelo de esta memoria, descrito en los Capítulos 4 y 5.

2.3.3. Transformers sobre observaciones irregulares y modelos híbridos

Más allá de los *encodings* continuos, una línea muy relevante del estado del arte reciente consiste en construir **tokens directamente a partir de observaciones irregulares**, evitando la regularización previa sobre una malla fija. En esta familia destacan, al menos, dos estrategias representativas.

La primera consiste en tratar la serie como un **conjunto de eventos u observaciones**. Un ejemplo claro es **STraTS** Tipirneni y Reddy, 2022, que representa cada medición mediante un triplete (t, f, v) compuesto por tiempo, variable y valor, y aplica un *Transformer* sobre estas observaciones embebidas. Su contribución principal es demostrar que un esquema de atención estándar puede operar directamente sobre datos clínicos dispersos e irregulares sin recurrir a imputación o agregación horaria agresiva. Además, incorpora un objetivo de auto-supervisión basado en *forecasting* para mejorar la calidad de las representaciones cuando los datos etiquetados son escasos. Este enfoque es conceptualmente cercano a la idea de esta memoria de trabajar con *timestamps* reales, aunque fue validado principalmente en tareas de clasificación clínica y con una tokenización centrada en observaciones individuales más que en ventanas de historia con *token objetivo*.

La segunda estrategia consiste en definir mecanismos de atención que respeten explícitamente la estructura irregular en los ejes temporal y multivariable. En esta línea, **CoFormer** Wei et al., 2023 modela cada muestra como un punto *variate-time* y alterna atención *intra-variate* e *inter-variate* sobre vecindarios definidos por variable y cercanía temporal. De esta forma, el modelo aprende por separado la evolución temporal de cada variable y las interacciones entre variables asincrónicas. Frente a un encoder *Transformer* completamente estándar, CoFormer introduce una inductive bias más fuerte y explícita para manejar la desalineación temporal; a cambio, su arquitectura es más específica del problema y menos directamente reutilizable como bloque genérico de *forecasting*.

Junto con estos modelos orientados a observaciones, siguen existiendo propuestas que integran nociones explícitas de **tiempo continuo** en la trayectoria latente o en la representación de la secuencia:

- Modelos que combinan *Transformers* con **Neural ODE** o **Neural CDE**, utilizando la atención para resumir subsecuencias o para parametrizar el campo vectorial de la EDO/ECDE, y resolviendo luego la dinámica en tiempo continuo.
- Propuestas como el **Rough Transformer**, que se apoyan en herramientas de *rough path theory* y firmas de caminos (*signatures*) para representar trayectorias irregulares, sobre las que se aplica atención. La idea central es mapear la serie irregular a un conjunto de características invariantes a reparametrizaciones del tiempo, que luego sirven como "tokens" de entrada al *Transformer* Moreno-Pino et al., 2024.

En conjunto, estos trabajos muestran que el panorama actual ya no se limita a "transformers regulares + imputación", sino que incluye arquitecturas capaces de operar directamente sobre observaciones asincrónicas. Sin embargo, persiste un compromiso claro entre **generalidad arquitectónica**, **sesgo inductivo específico para la irregularidad** y **simplicidad de implementación**. Precisamente ahí se sitúa la motivación de esta memoria: capturar la información temporal continua sin abandonar la estructura simple y reutilizable de un encoder *Transformer* convencional.

2.3.4. Agregación por *patches* y baselines simples para series irregulares

Muy recientemente han aparecido trabajos que replantean el *forecasting* de series irregulares desde una perspectiva de **agregación por bloques** (*patch aggregation*). Una propuesta representativa introduce un módulo de agregación consciente del tiempo, que agrupa observaciones irregulares en *patches* definidos sobre ventanas temporales y aplica operaciones de pooling o atenciones locales para producir una secuencia de tokens en una malla pseudo-regular. Esta secuencia agregada se procesa luego con redes relativamente simples (por ejemplo, MLP o bloques de atención ligeros) Liu et al., 2025.

De manera llamativa, estos métodos reportan que una combinación de:

- agregación temporal bien diseñada,
- modelos posteriores simples (incluso sin *Transformers* profundos),

puede competir o superar a arquitecturas mucho más complejas basadas en ODE/CDE o *Transformers* especializados Liu et al., 2025. Esto sugiere que, en series irregulares, **cómo se resumen los datos en el tiempo** puede ser tan importante como la arquitectura posterior.

La propuesta de esta memoria se alinea parcialmente con esta idea: en lugar de diseñar un *Transformer* radicalmente nuevo, se busca una combinación sencilla pero expresiva de:

- encoding temporal continuo sobre *timestamps* reales,
- construcción cuidadosa de ventanas de historia y *token target*,
- y un encoder *Transformer* estándar Vaswani et al., 2017.

De este modo, la irregularidad se aborda principalmente en la forma en que se representan y organizan los datos en la secuencia de entrada.

2.4. Clasificación de series de tiempo irregulares

Aunque el foco principal de esta memoria es el *forecasting*, muchas de las técnicas discutidas se han desarrollado inicialmente para tareas de **clasificación** sobre series irregulares, especialmente en el dominio de la salud. En esta sección se resume brevemente el estado del arte en clasificación, poniendo énfasis en su relación con los modelos ya descritos.

2.4.1. Métodos probabilísticos y kernels para series irregulares

Antes de la adopción masiva de deep learning, la clasificación de series irregulares se abordaba a menudo mediante:

- **Modelos de mezcla** de procesos gaussianos (por ejemplo, mezclas de GP o GP con kernels específicos por clase), donde la probabilidad de una trayectoria se evalúa bajo diferentes clases y se aplica la regla de Bayes Williams y Rasmussen, 2006.
- **Kernels dinámicos** que comparan series de distintos sujetos mediante alineamientos en tiempo continuo (por ejemplo, variantes de *dynamic time warping* continuo Müller, 2007; Sakoe y Chiba, 2003 o kernels de *time series shapelets* Hills et al., 2014; Ye y Keogh, 2009), aptos para métodos de clasificación kernelizados como SVM.

Estos enfoques explotan bien la estructura temporal y la irregularidad, pero pueden ser costosos en conjuntos de datos grandes y menos flexibles que los modelos neuronales para capturar relaciones complejas entre variables.

2.4.2. Modelos de deep learning para clasificación irregular

Buena parte de los modelos de deep learning revisados en la Sección 2.2 se introdujeron inicialmente para **clasificación** más que para forecasting. Ejemplos importantes incluyen:

- GRU-D y variantes con decaimientos y máscaras, aplicados a tareas como predicción de mortalidad, diagnóstico y detección temprana de sepsis Che et al., 2018.
- ODE-RNN, Latent ODE y GRU–ODE–Bayes, utilizados para modelar la evolución de estados fisiológicos latentes a partir de mediciones clínicas irregulares, con salidas de clasificación en tiempos específicos Chen et al., 2018; De Brouwer et al., 2019; Yulia et al., 2019.
- Neural CDE y modelos relacionados, que tratan la trayectoria como un camino continuo y aprenden funciones sobre dicho camino para predecir etiquetas globales o evento–tiempo Kidger et al., 2020.

En la práctica, las diferencias entre forecasting y clasificación se reducen a la elección de la capa de salida (regresión vs clasificación) y de la función de pérdida (MSE vs entropía cruzada). Las consideraciones arquitectónicas para manejar irregularidad son en gran medida las mismas.

2.4.3. Transformers para clasificación de series irregulares

El uso de *Transformers* para clasificación con observaciones irregulares ha madurado especialmente en el dominio biomédico, aunque la literatura sigue estando relativamente **fragmentada por tarea y dominio**. En términos generales, pueden distinguirse tres estrategias.

- **Preprocesamiento + Transformer estándar**: las series se regularizan mediante re-muestreo o imputación y se alimentan a un *Transformer* similar a los usados en series equiespaciadas, con una cabeza de clasificación sobre un token agregado.
- **Tokenización directa de observaciones irregulares**: en lugar de construir una malla temporal artificial, cada medición se convierte en un token o punto elemental. STraTS pertenece a esta familia y muestra que una arquitectura de atención aplicada a tripletes tiempo–variable–valor puede competir con baselines especializados en clasificación clínica irregular Tipirneni y Reddy, 2022. CoFormer también se sitúa en esta línea amplia, aunque con una atención más estructurada sobre vecindarios *intra/inter-variate* Wei et al., 2023.
- **Encodings temporales enriquecidos**: se mantienen los *timestamps* originales y se incorporan como encodings adicionales, por ejemplo mediante Time2Vec o embeddings sinusoidales continuos evaluados en tiempos reales Kazemi et al., 2019. Este planteamiento es el más cercano a la filosofía de la presente memoria, ya que busca preservar la forma del *Transformer* original modificando principalmente la representación temporal.

La evidencia reciente sugiere que los *Transformers* sí pueden ser competitivos en clasificación irregular cuando la representación temporal y la tokenización respetan la asincronía de las observaciones. No obstante, la mayor parte de los resultados más sólidos siguen concentrados en **clasificación clínica** y no necesariamente se trasladan de forma directa a problemas generales de *forecasting* multivariante irregular. Esta separación entre literatura de clasificación y literatura de pronóstico es una de las razones por las que sigue siendo valioso estudiar arquitecturas simples y transferibles entre ambas tareas.

2.5. Discusión y brechas de investigación

A partir de la revisión anterior, se pueden identificar varias brechas que motivan el modelo propuesto en esta memoria:

1. **Dependencia de re-muestreo e imputación**. Muchos enfoques prácticos siguen descansando en la regularización de la serie sobre una malla fija, con el riesgo de introducir sesgos y perder información sobre la distribución real de los tiempos de muestreo.
2. **Complejidad de algunos modelos en tiempo continuo**. Los modelos basados en ODE/CDE ofrecen un tratamiento elegante del tiempo continuo, pero a costa de mayor complejidad computacional y de implementación, lo que puede dificultar su uso en entornos de producción.

3. **Integración limitada entre *Transformers* y tiempo continuo para forecasting general**. Aunque existen propuestas relevantes como STraTS o CoFormer, la mayoría de los avances recientes se han validado sobre todo en clasificación irregular, frecuentemente en salud, o bien introducen mecanismos de atención muy específicos para el dominio. Siguen siendo relativamente pocas las arquitecturas que:
   - mantengan la estructura estándar de un encoder *Transformer* (auto-atención + bloques feedforward),
   - trabajen directamente con *timestamps* reales sin re-muestreo a una malla equiespaciada,
   - y permitan manejar de forma natural historias de longitud variable y horizontes de predicción heterogéneos en tareas de pronóstico multivariante.
4. **Baselines simples y comparabilidad**. Trabajos recientes orientados a series irregulares muestran que módulos de agregación temporal bien diseñados, combinados con modelos relativamente simples, pueden competir con arquitecturas profundas muy sofisticadas Liu et al., 2025. Esto pone de manifiesto la necesidad de:
   - modelos conceptualmente claros y modulares,
   - comparaciones experimentales cuidadosas frente a baselines robustos.

En este contexto, la contribución de la presente memoria se sitúa en el cruce de varias líneas:

- Diseñar un **encoding temporal continuo** que reciba directamente *timestamps* reales, inspirado en los posicional encodings sinusoidales clásicos, pero normalizado por una escala temporal configurable.
- Construir la tarea de forecasting mediante un **token objetivo** que representa explícitamente el instante de predicción dentro de la misma secuencia que la historia, permitiendo reutilizar un encoder *Transformer* estándar Vaswani et al., 2017.
- Explorar el entrenamiento con **historias de longitud variable** y **múltiples offsets de predicción** dentro de un mismo modelo, evaluando cómo estas decisiones de diseño afectan la capacidad de generalización en escenarios sintéticos y reales.

Los capítulos siguientes desarrollan en detalle esta propuesta: el Capítulo 3 formaliza el problema y la construcción de ejemplos supervisados; el Capítulo 4 presenta la arquitectura base de *Transformers* para series de tiempo; y el Capítulo 5 describe el modelo no equiespaciado propuesto, junto con su implementación concreta.

Capítulo 3

Series de tiempo y formulación del problema

En este capítulo se presentan los conceptos básicos relacionados con series de tiempo y se formula con precisión el problema que aborda esta memoria. El objetivo es establecer un marco formal que permita entender qué se espera del modelo propuesto y cómo se construyen los ejemplos supervisados a partir de datos temporales con marcas de tiempo no equiespaciadas.

Primero se introduce la notación para series de tiempo univariadas y multivariadas, junto con la distinción entre series equiespaciadas e irregulares. Luego se describe el problema de predicción a corto y mediano plazo como una tarea de aprendizaje supervisado, en la que el modelo recibe una *historia* (ventana de observaciones pasadas) y debe estimar el valor de la serie en uno o varios instantes futuros. Finalmente, se explica cómo se construyen los pares entrada–salida que se utilizarán durante el entrenamiento, incluyendo el rol de las longitudes de historia variables y de los offsets de predicción.

3.1. Conceptos básicos de series de tiempo

3.1.1. Definición y notación

De manera general, una **serie de tiempo** es una secuencia de observaciones ordenadas en el tiempo. En su forma más simple (caso univariado) se puede escribir como:

$$\{y_t\}_{t\in\mathcal{T}}, \tag{3.1}$$

donde $y_t \in \mathbb{R}$ es el valor observado en el tiempo t y $\mathcal{T}$ es el conjunto de instantes de observación.

En muchos problemas reales las observaciones son multivariadas. En ese caso se trabaja con vectores:

$$\{\mathbf{y}_t\}_{t\in\mathcal{T}}, \qquad \mathbf{y}_t \in \mathbb{R}^{d_{\text{out}}}, \tag{3.2}$$

donde cada componente de $\mathbf{y}_t$ puede corresponder, por ejemplo, a la temperatura, presión, flujo, etc. en el mismo instante temporal.

En esta memoria será útil distinguir entre:

- **Variables de entrada** (o *features*) denotadas por $\mathbf{x}_t \in \mathbb{R}^{d_{\text{in}}}$.
- **Variables objetivo** (o *targets*) denotadas por $\mathbf{y}_t \in \mathbb{R}^{d_{\text{out}}}$.

En algunos casos $\mathbf{x}_t$ y $\mathbf{y}_t$ pueden coincidir (se desea predecir la propia serie), mientras que en otros $\mathbf{x}_t$ y $\mathbf{y}_t$ representan magnitudes distintas.

De forma compacta, a lo largo del tiempo se dispondrá de una secuencia de tripletas:

$$\{(t_i, \mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^{T}, \tag{3.3}$$

con $t_1 < t_2 < \ldots < t_T$ los *timestamps* asociados a cada observación.

3.1.2. Series equiespaciadas vs. no equiespaciadas

En muchos modelos clásicos de series de tiempo (por ejemplo, ARIMA o ciertas arquitecturas recurrentes) se asume que los datos están muestreados en instantes equiespaciados:

$$t_i = t_0 + i \cdot \Delta t, \qquad \Delta t > 0 \text{ constante.} \tag{3.4}$$

En este caso suele ser suficiente trabajar con un índice entero i que representa el "paso de tiempo", y buena parte de la información temporal está implícita en la posición de la observación dentro de la secuencia.

Sin embargo, en muchos dominios aplicados los datos presentan:

- **Frecuencias de muestreo variables** (sensores que cambian de tasa según el modo de operación).
- **Datos faltantes** o ventanas en las que no se registran observaciones.
- **Eventos esporádicos** que no siguen una grilla temporal fija (por ejemplo, mediciones realizadas sólo ante ciertas condiciones).

En estos casos, los instantes de observación

$$t_1, t_2, \ldots, t_T \tag{3.5}$$

ya no están separados por un intervalo constante, y la información contenida en las diferencias $t_{i+1} - t_i$ pasa a ser relevante. A este tipo de datos nos referiremos como **series de tiempo no equiespaciadas** o **con timestamps irregulares**.

Trabajar directamente con timestamps reales permite, por ejemplo:

- Distinguir entre dos ejemplos con el mismo número de observaciones en la historia pero con escalas temporales distintas (historia concentrada en pocos minutos vs. varios días).
- Definir horizontes de predicción en términos de tiempo real (por ejemplo, "predecir a 15 minutos, 1 hora y 3 horas en el futuro") en lugar de solo "a 1, 3 o 5 pasos".

Estas características motivan la necesidad de modelos capaces de incorporar explícitamente la información temporal continua, lo que se abordará en capítulos posteriores mediante mecanismos de *encoding temporal continuo* dentro de la arquitectura Transformer.

3.2. Datos considerados en la memoria

La memoria considera dos escenarios generales de datos:

- **Datos sintéticos**, utilizados como entorno controlado para validar el comportamiento básico del pipeline y de la arquitectura en un contexto donde la generación de la serie puede analizarse con mayor trazabilidad.
- **Datos reales procesados**, organizados como una colección de datasets temporales independientes, sobre los cuales se ejecuta el benchmark principal reportado en los capítulos experimentales.

En ambos casos, la estructura de trabajo es esencialmente la misma: cada dataset contiene una columna temporal que fija el orden de observación y una o más variables numéricas que actúan como entradas y objetivos de predicción. En la etapa actualmente consolidada de esta memoria, el benchmark principal se apoya sobre datasets reales procesados con una variable objetivo escalar, de modo que el problema experimental activo corresponde a forecasting univariado con timestamps irregulares.

La razón para considerar tanto datos sintéticos como reales es metodológica. Los datos sintéticos permiten verificar que el pipeline de construcción de ventanas, escalado, entrenamiento y predicción funciona de extremo a extremo bajo condiciones más controladas. Los datos reales, en cambio, permiten evaluar si la arquitectura mantiene un comportamiento competitivo en escenarios más cercanos a una aplicación práctica, donde la irregularidad temporal y la variabilidad entre datasets son parte central del problema.

Los detalles específicos del protocolo experimental, de las configuraciones de entrenamiento y de la comparación entre modelos se desarrollan en el Capítulo 6. En este capítulo interesa principalmente dejar establecido que la formulación matemática introducida a continuación busca cubrir ambos escenarios bajo un mismo marco general.

3.3. Problema de predicción en series de tiempo

3.3.1. Predicción a partir de una ventana de historia

En la práctica, el problema de predicción se formula como una tarea de aprendizaje supervisado sobre ventanas finitas de la serie. Sea

$$\mathcal{H}_i = \{(t_j, \mathbf{x}_j, \mathbf{y}_j) : j \leq i\} \tag{3.6}$$

la historia completa disponible hasta el índice i. Dado un horizonte de predicción definido por un instante futuro $t^\star > t_i$, se desea construir un modelo

$$F_\theta : \mathcal{H}_i \longmapsto \widehat{\mathbf{y}}_{t^\star}, \tag{3.7}$$

parametrizado por θ, que entregue una predicción $\widehat{\mathbf{y}}_{t^\star}$ de la variable objetivo en $t^\star$.

Por motivos computacionales y de regularización, se trabaja con una **historia truncada** de longitud acotada. En esta memoria se utiliza una ventana deslizante que, para cada ejemplo, considera sólo las últimas h observaciones antes de $t^\star$:

$$\mathcal{H}_i^{(h)} = \{(t_j, \mathbf{x}_j, \mathbf{y}_j) : i - h + 1 \leq j \leq i\}. \tag{3.8}$$

En otras palabras, el modelo verá únicamente un fragmento reciente de la serie y deberá aprender a extraer de allí la información relevante para predecir en el horizonte deseado.

3.3.2. Horizontes de predicción y offsets

En el caso de datos equiespaciados, es habitual definir el horizonte de predicción en términos de pasos discretos, por ejemplo:

$$\text{predecir } y_{t+k} \text{ a partir de } \{y_{t-h+1}, \ldots, y_t\}, \tag{3.9}$$

donde k es el número de pasos hacia adelante.

En esta memoria se adopta una formulación que combina ambas visiones:

- A nivel de construcción del dataset, se define un **offset de predicción** o que indica cuántos índices después del *anchor* se tomará el target en la secuencia original.
- A nivel del modelo, además, se utilizan los timestamps reales t_i para informar la distancia temporal efectiva entre la última observación de la historia y el instante objetivo.

Más concretamente, para cada ejemplo se elige un índice *anchor* a (que marca el final de la historia) y un offset de predicción $o \geq 0$. El índice del target será entonces:

$$i_{\text{target}} = a + o. \tag{3.10}$$

El instante objetivo correspondiente es $t_{i_{\text{target}}}$ y el valor a predecir es $\mathbf{y}_{i_{\text{target}}}$.

Durante el entrenamiento se pueden considerar:

- Un único offset fijo (por ejemplo, siempre predecir un paso adelante).
- Un conjunto discreto de offsets posibles (por ejemplo $\{1, 3, 5\}$), que se muestren aleatoriamente. Esto permite entrenar al modelo para manejar distintos horizontes dentro de un mismo esquema.

3.4. Construcción de ejemplos supervisados

3.4.1. Ventanas deslizantes y tamaño de la historia

Sea T el número total de observaciones disponibles. Dado un **tamaño máximo de historia** H y un **stride** $s \geq 1$, se generan ejemplos recorriendo la serie con una ventana deslizante. De manera simplificada, los índices *anchor* se pueden escribir como:

$$a \in \{H, H + s, H + 2s, \ldots, a_{\text{máx}}\}, \tag{3.11}$$

donde $a_{\text{máx}}$ se elige de modo que exista al menos un offset de predicción válido (es decir, $a_{\text{máx}} + o_{\text{máx}} \leq T - 1$, con $o_{\text{máx}}$ el máximo offset considerado).

Para cada anchor a se define una historia formada por las últimas h observaciones anteriores a a:

$$\mathcal{H}_a^{(h)} = \{(t_j, \mathbf{x}_j, \mathbf{y}_j)\}_{j=a-h}^{a-1}, \tag{3.12}$$

donde h puede tomar valores entre una longitud mínima $H_{\text{mín}}$ y la longitud máxima H:

$$H_{\text{mín}} \leq h \leq H. \tag{3.13}$$

En esta memoria se consideran dos configuraciones:

- **Historia fija**: $H_{\text{mín}} = H$, por lo que todas las historias tienen la misma longitud.
- **Historia variable**: $H_{\text{mín}} < H$, y para cada ejemplo se muestrea aleatoriamente una longitud $h \in [H_{\text{mín}}, H]$. De este modo, el modelo se entrena para manejar ventanas de contexto de tamaño variable, algo relevante cuando la densidad temporal de los datos cambia en el tiempo.

3.4.2. Definición de la entrada y la salida de cada ejemplo

Cada ejemplo supervisado queda entonces determinado por:

$$\text{Entrada} : \left(\{t_j\}_{j=a-h}^{a-1}, \{\mathbf{x}_j\}_{j=a-h}^{a-1}, \{\mathbf{y}_j\}_{j=a-h}^{a-1}, t_{i_{\text{target}}}\right), \tag{3.14}$$

$$\text{Salida} : \mathbf{y}_{i_{\text{target}}}, \tag{3.15}$$

donde $i_{\text{target}} = a + o$ como se describió antes.

En términos más operativos, es útil separar:

- Un vector de tiempos de la historia:

$$\mathbf{t}^{(\text{past})} = (t_{a-h}, \ldots, t_{a-1}),$$

- Una matriz de valores de entrada:

$$X^{(\text{past})} \in \mathbb{R}^{h \times d_{\text{in}}},$$

- El timestamp objetivo $t^{(\text{target})} = t_{i_{\text{target}}}$,
- Y el vector de salida

$$\mathbf{y}^{(\text{target})} = \mathbf{y}_{i_{\text{target}}} \in \mathbb{R}^{d_{\text{out}}}.$$

El modelo propuesto en esta memoria recibirá como entrada una secuencia construida a partir de la historia y del timestamp objetivo (mediante la adición de un token especial), y deberá producir una estimación $\widehat{\mathbf{y}}^{(\text{target})}$ de la salida verdadera. Los detalles de esta representación se desarrollan en el capítulo dedicado al modelo Transformer.

3.5. Formulación como problema de aprendizaje supervisado

3.5.1. Función objetivo

Sea $\mathcal{D}_{\text{train}}$ el conjunto de ejemplos construidos según el procedimiento anterior. El problema de entrenamiento consiste en hallar parámetros θ que minimicen una función de pérdida agregada sobre todos los ejemplos de entrenamiento.

Para problemas de regresión se utilizará principalmente la pérdida de error cuadrático medio:

$$\mathcal{L}(\theta) = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(\text{entrada},\mathbf{y})\in\mathcal{D}_{\text{train}}} \|F_\theta(\text{entrada}) - \mathbf{y}\|^2 , \tag{3.16}$$

donde F_θ representa al modelo (en este caso, el Transformer especializado) y la norma se aplica sobre las componentes de la variable objetivo.

En el capítulo de entrenamiento se considerarán también otras métricas de evaluación, como error absoluto medio (MAE), raíz del error cuadrático medio (RMSE) y error porcentual absoluto medio (MAPE), que permiten interpretar la calidad de las predicciones en distintas unidades y escalas.

3.5.2. Entrenamiento, validación y prueba

Para estimar la capacidad de generalización del modelo —es decir, su desempeño sobre datos no vistos durante el entrenamiento— se divide la serie en tres subconjuntos temporales disjuntos:

- **Entrenamiento**: segmento inicial de la serie, usado para ajustar los parámetros del modelo.
- **Validación**: segmento intermedio, usado para seleccionar hiperparámetros (por ejemplo, tamaño de historia, offsets de predicción, arquitectura del modelo) y para aplicar criterios de *early stopping*.
- **Prueba**: segmento final, reservado para evaluar el desempeño del modelo una vez fijados los hiperparámetros.

La división se realiza respetando el orden temporal de los datos, de modo que el modelo siempre se entrene en el pasado y se evalúe en el futuro, evitando fugas de información temporal.

3.6. Resumen del capítulo

En este capítulo se han establecido los elementos necesarios para describir con precisión el problema abordado en la memoria:

- Se introdujo la notación básica para series de tiempo univariadas y multivariadas, distinguiendo explícitamente entre variables de entrada $\mathbf{x}_t$, variables objetivo $\mathbf{y}_t$ y timestamps t.

- Se discutió la diferencia entre series equiespaciadas y no equiespaciadas, destacando las motivaciones para trabajar directamente con marcas de tiempo irregulares.
- Se describió, a nivel general, el tipo de datos considerados en la memoria, distinguiendo entre escenarios sintéticos de validación controlada y datasets reales procesados usados en el benchmark principal.
- Se formuló el problema de predicción como una tarea de aprendizaje supervisado basada en ventanas de historia finita y horizontes de predicción definidos mediante offsets sobre la secuencia original y timestamps reales.
- Se describió el procedimiento para construir ejemplos supervisados a partir de la serie completa, incluyendo la posibilidad de usar longitudes de historia variables y un conjunto discreto de offsets de predicción.
- Se presentó la función objetivo de entrenamiento y el esquema de partición temporal en conjuntos de entrenamiento, validación y prueba.

Sobre este marco se construirá, en los capítulos siguientes, la arquitectura Transformer propuesta para manejar timestamps no equiespaciados y realizar predicciones a distintos horizontes, junto con los experimentos que evalúan su desempeño.

Capítulo 4

Transformers para series de tiempo no equiespaciadas

En este capítulo se presenta la arquitectura *Transformer* adaptada al contexto de series de tiempo no equiespaciadas que se utilizará en esta memoria. El objetivo es describir con precisión cómo se representan las ventanas de historia, cómo se incorporan los *timestamps* irregulares dentro del modelo y cómo se formula la tarea de predicción como una operación sobre un *token target* específico.

Primero se revisa brevemente la arquitectura Transformer clásica como modelo de atención sobre secuencias. A continuación se discuten sus ventajas y desafíos al trabajar con series de tiempo. Finalmente, se introduce el modelo propuesto, basado en un *encoder* Transformer con un esquema de *token target* y un *encoding* temporal continuo que opera directamente sobre timestamps reales.

4.1. Recordatorio de la arquitectura Transformer

El Transformer Vaswani et al., 2017 es una arquitectura originalmente propuesta para tareas de procesamiento de lenguaje natural, donde la entrada es una secuencia de *tokens* discretos. Cada token x_i se proyecta a un vector denso $\mathbf{e}_i \in \mathbb{R}^{d_{\text{model}}}$ mediante una tabla de embeddings, y se le suma un *positional encoding* que entrega información sobre su posición en la secuencia. Sobre esta representación se aplican varias capas de:

- **Atención multi-cabeza** (*Multi-Head Self-Attention*), que mezcla información entre todas las posiciones.
- **Bloques feedforward** independientes por posición.
- **Conexiones residuales** y normalización de capa (*LayerNorm*).

En cada capa, la atención utiliza tres proyecciones lineales de los vectores de entrada:

$$Q = XW^Q, \qquad K = XW^K, \qquad V = XW^V, \tag{4.1}$$

donde $X \in \mathbb{R}^{L \times d_{\text{model}}}$ es la matriz de representaciones de la secuencia, L es la longitud de la secuencia y W^Q, W^K, W^V son matrices de pesos. La atención escalar producto se computa como:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right) V, \tag{4.2}$$

donde d_h es la dimensión de cada cabeza. En la versión multi-cabeza se repite este cálculo en paralelo para varias proyecciones y se concatenan los resultados.

En esta memoria se trabaja con un **encoder Transformer** únicamente (sin decodificador), tal como se detalla en el Anexo A. Este encoder recibe una secuencia de vectores ya embebidos, aplica varias capas de auto-atención y redes feedforward, y devuelve una representación contextualizada para cada posición de la secuencia.

4.2. Transformers en series de tiempo

4.2.1. Ventajas

Aplicar Transformers a series de tiempo presenta varias ventajas:

- **Dependencias de largo alcance**: la auto-atención permite que cualquier posición atienda directamente a cualquier otra, independientemente de la distancia temporal, lo que facilita capturar patrones de largo plazo.
- **Paralelización**: a diferencia de modelos autoregresivos estrictos (como ciertas redes recurrentes), todas las posiciones de la historia pueden procesarse en paralelo, lo que resulta eficiente en hardware moderno.
- **Flexibilidad en la longitud de la secuencia**: el mismo conjunto de pesos puede operar sobre secuencias de longitud variable, siempre que la dimensión de los vectores de entrada se mantenga constante.

Estas características son especialmente atractivas para problemas donde las ventanas de historia pueden tener tamaños distintos y donde se desea manejar horizontes de predicción múltiples, como se describió en el Capítulo 3.

4.2.2. Desafíos con timestamps irregulares

Los Transformers clásicos asumen que la información de *posición* se puede representar mediante índices enteros $1, 2, \ldots, L$, y utilizan un encoding posicional (típicamente sinusoidal) definido sobre esos índices. Sin embargo, en series de tiempo no equiespaciadas:

- La separación temporal entre observaciones consecutivas, $t_{i+1} - t_i$, no es constante.
- Puede haber lagunas de observaciones o períodos de muestreo más denso.
- El horizonte de predicción puede definirse naturalmente en unidades de tiempo físico (segundos, minutos, horas) y no sólo en "pasos" discretos.

Si se ignorara esta irregularidad y se tratara cada observación como si correspodiera a un paso fijo, el modelo podría confundir historias que, pese a tener el mismo número de puntos, se desarrollan en escalas de tiempo muy distintas. Por ello, en esta memoria se propone incorporar explícitamente el timestamp real en el encoding temporal del Transformer.

4.3. Diseño general del modelo propuesto

El modelo propuesto se basa en un **encoder Transformer** especializado para series de tiempo, al que se denominará *TimeSeriesTransformer*. Su estructura general se puede describir en los siguientes pasos:

1. A partir de la serie completa se construyen ejemplos supervisados mediante ventanas de historia y offsets de predicción (Capítulo 3).
2. Para cada ejemplo se genera una secuencia de entrada que incluye:
   - Los valores históricos y sus timestamps.
   - Un **token target** adicional, asociado al timestamp objetivo que se quiere predecir.
3. Cada posición de la secuencia se representa mediante la suma de tres componentes:
   - Un *embedding de valores* (variables de entrada).
   - Un *encoding temporal continuo* basado en el timestamp.
   - Un *embedding de flag* que distingue entre tokens de historia y token target.
4. La secuencia resultante se procesa con un encoder Transformer estándar de varias capas.
5. La representación final en la posición del token target se pasa a una *cabeza de regresión* que produce la predicción para el instante objetivo.

En las siguientes secciones se detallan cada uno de estos componentes, conectándolos con la implementación concreta usada en los experimentos.

4.4. Construcción de la secuencia con token target

Sea un ejemplo supervisado definido por:

- Una historia de longitud L:

$$X^{(\text{past})} = \begin{bmatrix} \mathbf{x}_{t_{a-L}}^\top \\ \vdots \\ \mathbf{x}_{t_{a-1}}^\top \end{bmatrix} \in \mathbb{R}^{L \times d_{\text{in}}}, \qquad \mathbf{t}^{(\text{past})} = (t_{a-L}, \ldots, t_{a-1}) \in \mathbb{R}^L,$$

- Un timestamp objetivo $t^\star = t_{i_{\text{target}}}$ y el valor a predecir $\mathbf{y}^\star = \mathbf{y}_{i_{\text{target}}} \in \mathbb{R}^{d_{\text{out}}}$.

El **constructor de secuencias** (`SequenceBuilder`) genera una secuencia extendida de longitud $L+1$ añadiendo al final un token especial asociado al timestamp objetivo:

$$\tilde{X} = \begin{bmatrix} X^{(\text{past})} \\ \mathbf{x}^{(\text{target})\top} \end{bmatrix} \in \mathbb{R}^{(L+1) \times d_{\text{in}}}, \tag{4.3}$$

$$\tilde{\mathbf{t}} = (\mathbf{t}^{(\text{past})},\ t^\star) \in \mathbb{R}^{L+1}, \tag{4.4}$$

donde $\mathbf{x}^{(\text{target})}$ es un vector *placeholder* para el token target. En los experimentos de esta memoria se considera, principalmente, la opción:

$$\mathbf{x}^{(\text{target})} = \mathbf{0}, \tag{4.5}$$

es decir, un vector de ceros que indica que no se conocen (ni se usan) variables de entrada en el instante objetivo.

Además, se construye un vector booleano que marca la posición del token target:

$$\mathbf{f} = (0, \ldots, 0, 1) \in \{0, 1\}^{L+1}, \tag{4.6}$$

donde el valor 1 en la última posición indica que se trata del token target. En la implementación este vector se llama `is_target_mask`.

Es importante destacar que este flag **no** codifica por sí mismo el instante que se desea predecir. La fecha objetivo queda determinada por el timestamp asociado al token target, es decir, por el último elemento de $\tilde{\mathbf{t}}$. Por lo tanto, dos ejemplos pueden tener exactamente el mismo patrón de flags y, sin embargo, pedir predicciones para fechas distintas; la diferencia entre ambos casos entra al modelo a través del encoding temporal evaluado en $t^\star$.

Esta secuencia extendida $\{(\tilde{\mathbf{x}}_i, \tilde{t}_i, f_i)\}_{i=1}^{L+1}$ es la que se entregará al modelo Transformer.

4.5. Embeddings y encodings

Cada posición de la secuencia se convierte en un vector de dimensión d_{model} mediante la suma de tres componentes:

$$\mathbf{h}_i^{(0)} = \underbrace{E_{\text{val}}(\tilde{\mathbf{x}}_i)}_{\text{embedding de valores}} + \underbrace{E_{\text{time}}(\tilde{t}_i)}_{\text{encoding temporal}} + \underbrace{E_{\text{flag}}(f_i)}_{\text{flag historia/target}}, \qquad i = 1, \ldots, L+1. \tag{4.7}$$

4.5.1. Embedding de valores

El *embedding de valores* (`FeatureEmbedding`) es una proyección lineal que lleva el vector de entrada $\tilde{\mathbf{x}}_i \in \mathbb{R}^{d_{\text{in}}}$ a la dimensión interna del modelo:

$$E_{\text{val}}(\tilde{\mathbf{x}}_i) = W_{\text{val}}\tilde{\mathbf{x}}_i + \mathbf{b}_{\text{val}}, \qquad W_{\text{val}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{in}}},\ \mathbf{b}_{\text{val}} \in \mathbb{R}^{d_{\text{model}}}. \tag{4.8}$$

Opcionalmente se puede aplicar una normalización de capa sobre la dimensión d_{model}, lo que se controla mediante un parámetro de configuración (`use_layernorm`).

4.5.2. Encoding temporal continuo

El *encoding temporal* (`TimePositionalEncoding`) tiene por objetivo incorporar de manera explícita el timestamp real en la representación del token. En lugar de usar un índice discreto $1, \ldots, L+1$, se trabaja directamente con el tiempo físico.

Sea $\tilde{\mathbf{t}} = (\tilde{t}_1, \ldots, \tilde{t}_{L+1})$ el vector de timestamps de la secuencia extendida. Para cada secuencia se define un tiempo relativo normalizado:

$$\tau_i = \frac{\tilde{t}_i - \tilde{t}_1}{s_{\text{time}}}, \qquad i = 1, \ldots, L+1, \tag{4.9}$$

donde $s_{\text{time}} > 0$ es un hiperparámetro de escala temporal (`time_scale`) que controla cuántas unidades de tiempo físico corresponden aproximadamente a un "paso" en el encoding. Por ejemplo, si los timestamps se expresan en segundos y $s_{\text{time}} = 900$, entonces un salto $\tau_i - \tau_j = 1$ equivale a 15 minutos.

Sobre τ_i se aplica un encoding sinusoidal continuo, siguiendo la idea de Vaswani et al., 2017 pero usando tiempos reales:

$$\big(E_{\text{time}}(\tilde{t}_i)\big)_{2k} = \sin\left(\frac{\tau_i}{10000^{\frac{2k}{d}}}\right), \tag{4.10}$$

$$\big(E_{\text{time}}(\tilde{t}_i)\big)_{2k+1} = \cos\left(\frac{\tau_i}{10000^{\frac{2k}{d}}}\right), \tag{4.11}$$

para $k = 0, 1, \ldots, \lfloor \frac{d}{2} \rfloor - 1$. De este modo, diferencias en el tiempo físico se traducen en rotaciones dentro de un espacio sinusoidal.

La implementación también permite, opcionalmente, reemplazar este encoding por un MLP sobre τ_i (`mode = "mlp"`), aprendiendo directamente una proyección no lineal desde el tiempo a $\mathbb{R}^{d_{\text{model}}}$.

4.5.3. Embedding de flag historia/target

El tercer componente es un *embedding de flag* (`TargetFlagEmbedding`) que diferencia explícitamente entre:

- Tokens de **historia** ($f_i = 0$).
- El token **target** ($f_i = 1$).

Este embedding se implementa mediante una tabla de tamaño $2 \times d_{\text{model}}$:

$$E_{\text{flag}}(f_i) = \mathbf{e}^{(\text{flag})}_{f_i}, \tag{4.12}$$

de manera que el modelo pueda aprender patrones específicos asociados a la posición objetivo (por ejemplo, el hecho de que ese token no contiene valores observados sino el timestamp donde se desea predecir).

En otras palabras, E_{flag} sólo comunica el *rol* del token dentro de la secuencia (historia u objetivo), mientras que la *fecha concreta* a la que corresponde ese objetivo queda codificada en $E_{\text{time}}(t^\star)$. Esta separación es precisamente la que permite reutilizar la misma arquitectura para distintos horizontes de predicción sin cambiar la posición relativa del token target dentro de la secuencia.

4.6. Encoder Transformer y máscara de atención

Una vez construidas las representaciones iniciales $\mathbf{h}^{(0)}_1, \ldots, \mathbf{h}^{(0)}_{L+1}$, se apilan en una matriz

$$H^{(0)} \in \mathbb{R}^{(L+1)\times d_{\text{model}}}, \tag{4.13}$$

que se procesa mediante N bloques encoder Transformer idénticos:

$$H^{(\ell+1)} = \text{EncoderBlock}^{(\ell)}\big(H^{(\ell)}\big), \qquad \ell = 0, \ldots, N-1. \tag{4.14}$$

Cada bloque encoder (`TransformerEncoderBlock`) utiliza:

- Una capa de auto-atención multi-cabeza (`MultiHeadSelfAttention`).
- Una red feedforward posición a posición.
- Conexiones residuales y *LayerNorm*.

4.6.1. Máscara de padding y máscara causal opcional

Dado que las longitudes de historia pueden variar entre ejemplos, al construir los *batches* se utilizan tensores con padding. Para ello se define una **máscara de padding** $\mathbf{M}_{\text{pad}} \in \{0,1\}^{B\times(L_{\text{máx}}+1)}$, donde B es el tamaño de batch y $L_{\text{máx}}$ la longitud máxima del batch. Un valor 1 indica que la posición correspondiente es padding (y, por tanto, no debe ser atendida ni contribuir a los cálculos de atención). Esta máscara se pasa a la capa de atención como `key_padding_mask`.

Adicionalmente, el modelo admite una **máscara causal** opcional (`create_causal_mask`). Cuando está activa, se prohíbe que una posición atienda a posiciones futuras, imponiendo una estructura autoregresiva. En los experimentos principales se utiliza atención completa (sin máscara causal) porque el token target se encuentra al final de la secuencia y lo que se desea es que pueda recibir información de toda la historia.

4.7. Token target y cabeza de regresión

Tras aplicar las N capas encoder se obtiene una representación contextualizada para cada posición:

$$H^{(N)} = \begin{bmatrix} \mathbf{z}_1^\top \\ \vdots \\ \mathbf{z}_{L+1}^\top \end{bmatrix} \in \mathbb{R}^{(L+1)\times d_{\text{model}}}. \tag{4.15}$$

El índice del token target corresponde a la última posición de la secuencia (o, en general, a la posición donde $f_i = 1$). Denotemos su representación por:

$$\mathbf{z}_{\text{target}} = \mathbf{z}_{L+1} \in \mathbb{R}^{d_{\text{model}}}. \tag{4.16}$$

La **cabeza de regresión** (`RegressionHead`) proyecta este vector al espacio de salida:

$$\widehat{\mathbf{y}}^\star = g_{\theta_{\text{head}}}(\mathbf{z}_{\text{target}}), \tag{4.17}$$

donde $g_{\theta_{\text{head}}}$ es, en su forma más simple, una capa lineal:

$$\widehat{\mathbf{y}}^\star = W_{\text{out}}\mathbf{z}_{\text{target}} + \mathbf{b}_{\text{out}}, \qquad W_{\text{out}} \in \mathbb{R}^{d_{\text{out}}\times d_{\text{model}}}, \tag{4.18}$$

aunque la implementación permite incluir una pequeña MLP (una capa oculta adicional con función de activación y dropout) si se desea mayor capacidad.

El entrenamiento se realiza minimizando la pérdida entre $\widehat{\mathbf{y}}^\star$ y la salida verdadera $\mathbf{y}^\star$, típicamente usando el error cuadrático medio, tal como se describe en detalle en el capítulo de entrenamiento y experimentos.

4.8. Manejo de historias y horizontes variables

Una de las motivaciones centrales de este diseño es permitir que el modelo:

1. Trabaje con historias de longitud variable, adaptándose a series con densidades de muestreo distintas.
2. Soporte distintos horizontes de predicción (diferentes offsets) dentro de un mismo esquema arquitectónico.

4.8.1. Historias de longitud variable

El tamaño máximo de historia H se fija como hiperparámetro, pero durante el entrenamiento se puede muestrear una longitud h entre un mínimo $H_{\text{mín}}$ y H. Esto significa que el modelo ve ejemplos con secuencias de entrada de distinto largo, lo que lo entrena a ser robusto frente a ventanas de contexto más cortas o más largas.

Gracias a la naturaleza del Transformer:

- No es necesario reconfigurar pesos cuando cambia h; sólo se modifica la longitud de la secuencia.
- La máscara de padding garantiza que, dentro de un mismo batch, las posiciones de padding no afecten la atención ni la salida.

En términos prácticos, esto se implementa en el `TimeSeriesDataset` (que elige h y el offset) y en el `collate_fn` que construye los tensores de batch con padding.

4.8.2. Offsets de predicción múltiples

El conjunto de offsets de predicción puede definirse como:

$$\mathcal{O} = \{o_1, o_2, \ldots, o_K\}, \tag{4.19}$$

y para cada ejemplo de entrenamiento se elige un offset $o \in \mathcal{O}$ (por ejemplo, al azar). El timestamp objetivo $t^\star$ y, en consecuencia, el token target, cambian entonces según el horizonte seleccionado.

Este mecanismo tiene dos efectos:

- Permite entrenar un solo modelo capaz de predecir a distintos horizontes (por ejemplo, 15 minutos, 1 hora y 3 horas) sin necesidad de entrenar un modelo separado por cada horizonte.
- Obliga al modelo a utilizar de manera explícita la información temporal contenida en el encoding E_{time}, ya que el instante objetivo no está determinado por el flag binario ni por la posición final del token, sino por el timestamp que se le asigna y que luego se transforma en su encoding temporal.

En la práctica, el uso de offsets múltiples se configura a través del parámetro `target_offset_choices` en el dataset, descrito en el Capítulo 3.

4.9. Inferencia y pronósticos multi-step

Para la fase de inferencia se ofrecen dos componentes de alto nivel:

- **Predictor**, que envuelve al modelo Transformer y se encarga de:
  - Aplicar los *scalers* de entrada y salida (normalización y des-normalización).
  - Construir la secuencia con token target para un único timestamp objetivo.
- **RollingForecaster**, que permite realizar pronósticos multi-step sobre un conjunto de timestamps futuros.

Dado un historial $(X^{(\text{past})}, \mathbf{t}^{(\text{past})})$ y una colección de timestamps futuros $\{t_1^\star, \ldots, t_M^\star\}$, el `RollingForecaster` llama repetidamente al `Predictor` para obtener:

$$\widehat{\mathbf{y}}_1^\star, \ldots, \widehat{\mathbf{y}}_M^\star, \tag{4.20}$$

manteniendo fija la historia y cambiando sólo el token target. Este esquema corresponde al modo "`fixed_history`"; en futuras extensiones se podría considerar un modo autoregresivo que vaya incorporando las predicciones al historial.

4.10. Resumen del capítulo

En este capítulo se ha descrito la arquitectura Transformer adaptada para series de tiempo no equiespaciadas utilizada en esta memoria. Los elementos clave son:

- El uso de un **encoder Transformer** sobre una secuencia que incluye un **token target** asociado al instante objetivo de predicción.
- La construcción de la secuencia mediante el `SequenceBuilder`, que concatena historia y token target, junto con una máscara `is_target_mask`.
- Un esquema de **encoding temporal continuo** (`TimePositionalEncoding`) que opera directamente sobre los timestamps, permitiendo manejar datos no equiespaciados y distintos horizontes de predicción.
- La combinación de un **embedding de valores**, un **embedding de flag historia/target** y el encoding temporal para representar cada token en la dimensión interna del modelo.
- El uso de máscaras de padding y, opcionalmente, máscaras causales, para manejar secuencias de longitud variable dentro de los bloques de auto-atención.
- Una **cabeza de regresión** sobre la representación del token target, que produce la predicción en el instante deseado.
- Herramientas de inferencia (`Predictor`, `RollingForecaster`) que facilitan la aplicación del modelo a pronósticos de uno o múltiples pasos en el tiempo.

En el capítulo siguiente se describirá el procedimiento de entrenamiento, los conjuntos de datos utilizados y el diseño de los experimentos que permitirán evaluar empíricamente el desempeño de este modelo frente a otros enfoques basados en Transformers y modelos baselines para series de tiempo.

Capítulo 5

Modelo propuesto para series de tiempo no equiespaciadas

En este capítulo se describe en detalle el modelo propuesto para abordar la predicción de series de tiempo *no equiespaciadas* utilizando una arquitectura tipo Transformer. El diseño busca mantener, en la medida de lo posible, la estructura y ventajas del Transformer original, introduciendo modificaciones específicas para tratar directamente con *timestamps* irregulares sin necesidad de re-muestrear la serie sobre una malla uniforme.

Recordando la notación del Capítulo 3, consideremos una serie temporal multivariada observada en instantes

$$t_1 < t_2 < \ldots < t_T, \quad t_i \in \mathbb{R},$$

no necesariamente equiespaciados, y valores observados

$$x_i = x(t_i) \in \mathbb{R}^D, \quad i = 1, \ldots, T.$$

El objetivo del modelo es aprender una función

$$f_\theta : \big(\{(t_{i-h+1}, x_{i-h+1}), \ldots, (t_i, x_i)\}, t^\star\big) \mapsto \hat{y}(t^\star),$$

capaz de predecir el valor futuro (o pasado) en un *timestamp* objetivo $t^\star$, a partir de una historia de longitud variable h.

La implementación concreta se realiza mediante un módulo `TimeSeriesTransformer`, que combina:

- un *embedding* de valores continuos (`FeatureEmbedding`),
- un *encoding temporal continuo* basado en *timestamps* reales (`TimePositionalEncoding`),
- un *flag* para distinguir el token de historia del token objetivo (`TargetFlagEmbedding`),
- un *encoder Transformer* estándar sobre la secuencia enriquecida,
- y una cabeza de regresión sobre el token objetivo (`RegressionHead`).

En las secciones siguientes se detalla cada uno de estos componentes y cómo interactúan con el flujo de datos definido en el Capítulo 3.

5.1. Objetivos de diseño

El modelo propuesto se construyó alrededor de los siguientes objetivos de diseño:

1. **Respetar la irregularidad temporal**: evitar, en la medida de lo posible, el remuestreo a una malla equiespaciada, trabajando directamente con los *timestamps* originales.

2. **Separar claramente valor y tiempo**: representar por separado las variables de entrada (temperatura, presión, etc.) y la información temporal, combinándolas recién a nivel de *embedding*.

3. **Mantener compatibilidad con el Transformer estándar**: reutilizar la mayor parte de la estructura de atención multi-cabeza y bloques encoder descritos en el Capítulo 4, de modo que las extensiones sean lo menos invasivas posible.

4. **Manejar historias de longitud variable y horizontes de predicción heterogéneos**: permitir que, durante el entrenamiento, el modelo vea ejemplos con distintas longitudes de historia y con diferentes *offsets de predicción*, con el fin de mejorar su robustez.

5. **Soportar fácilmente inferencia multi-step**: a través de una capa de inferencia (`Predictor` y `RollingForecaster`) que oculte los detalles de construcción de secuencias y des-escalado.

5.2. Representación de entradas y construcción de ejemplos

5.2.1. Separación entre valores y timestamps

A nivel de datos crudos, el modelo recibe:

- un arreglo de *timestamps* $(t_1, \ldots, t_T)$ ya ordenado,
- una matriz de valores $V \in \mathbb{R}^{T \times D_{\text{total}}}$ que contiene:
  - las primeras D_{in} columnas como variables de entrada (*features*),
  - las siguientes D_{out} como variables objetivo (*targets*), cuando no se especifica un arreglo de *targets* separado.

Esta organización se materializa en la clase `TimeSeriesDataset`, que recibe:

- `values`: matriz $[T, D_{\text{total}}]$,
- `timestamps`: vector $[T]$,
- los parámetros `input_dim` y `output_dim` para separar entrada y salida.

El preprocesamiento descrito en el Capítulo 3 (normalización `StandardScaler`, splits temporales, etc.) se aplica antes de construir el `TimeSeriesDataset`.

5.2.2. Ventanas de historia y targets en TimeSeriesDataset

Cada ejemplo del `TimeSeriesDataset` se construye a partir de tres elementos clave:

1. **Longitud máxima de historia** (`history_length`, $h_{\text{máx}}$): número máximo de puntos pasados considerados.

2. **Longitud mínima de historia** (`min_history_length`, $h_{\text{mín}}$): permite muestrear historias de longitud variable entre $h_{\text{mín}}$ y $h_{\text{máx}}$.

3. **Offset(s) de predicción** (`target_offset` o `target_offset_choices`): distancia (en índices) entre el *anchor* de la historia y el índice del target.

Para cada ejemplo se elige un índice *anchor* i tal que exista suficiente historia hacia atrás y suficiente espacio hacia adelante para el target. De forma simplificada:

$$i \in \{h_{\text{máx}}, h_{\text{máx}} + \texttt{stride}, \ldots, T - 1 - o_{\text{máx}}\},$$

donde $o_{\text{máx}}$ es el máximo *offset* posible tomado de `target_offset_choices` (o de `target_offset` si sólo se usa un valor fijo).

Dado un *anchor* i:

- se muestrea una longitud de historia $h \in [h_{\text{mín}}, h_{\text{máx}}]$ (si `min_history_length` es distinto de `history_length`, de lo contrario $h = h_{\text{máx}}$),

- se define la historia como

$$\{(t_{i-h+1}, x_{i-h+1}), \ldots, (t_i, x_i)\},$$

- se elige un *offset* de predicción o desde `target_offset_choices` y se define el índice objetivo

$$j = i + o,$$

- el *timestamp* objetivo es $t^\star = t_j$ y el valor objetivo es $y^\star = y_j$.

El resultado que retorna `TimeSeriesDataset` para cada ejemplo es un diccionario con:

- `past_values`: matriz $[h, D_{\text{in}}]$,

- `past_timestamps`: vector $[h]$,

- `target_timestamp`: escalar $t^\star$,

- `target_values`: vector $[D_{\text{out}}]$.

Esta representación todavía no está en el formato que consume el Transformer; para ello se introduce el `SequenceBuilder`.

5.2.3. Construcción de secuencias con token objetivo

El `SequenceBuilder` es responsable de transformar el ejemplo anterior en la secuencia final que se alimenta al modelo. Conceptualmente, se añade un *token objetivo* al final de la historia:

$$\underbrace{(t_{i-h+1}, x_{i-h+1})}_{\text{historia}}, \ldots, \quad \underbrace{(t_i, x_i)}_{\text{último de la historia}} \quad , \quad \underbrace{(t^\star, \tilde{x}^\star)}_{\text{token objetivo}}$$

donde $\tilde{x}^\star$ es un *placeholder* para los valores del token objetivo. En la implementación se ofrecen dos opciones:

- `"zeros"`: $\tilde{x}^\star$ es el vector de ceros (no se aporta información de valor al token objetivo).
- `"last"`: $\tilde{x}^\star$ copia el último vector de la historia (x_i).

Después de esta operación, el `SequenceBuilder` devuelve:

- `input_values`: matriz $[L, D_{\text{in}}]$ con $L = h + 1$ (historia + token objetivo),
- `input_timestamps`: vector $[L]$ con los *timestamps* de historia y $t^\star$,
- `is_target_mask`: vector booleano $[L]$ con `True` sólo en la posición del token objetivo,
- `target_values`, `target_timestamp`: iguales a los del `TimeSeriesDataset`.

La idea central es que el token objetivo queda definido por la combinación de una máscara que identifica su rol y de su *timestamp*. La máscara sólo indica que esa posición debe interpretarse como una consulta de predicción; el timestamp fija la fecha concreta a la que apunta esa consulta. Dicho de otro modo, el horizonte de predicción no se controla con un identificador discreto adicional, sino asignando al token objetivo el timestamp $t^\star$ y dejando que `TimePositionalEncoding` transforme ese tiempo en una representación continua.

El módulo `build_collate_fn` se encarga posteriormente de agrupar estos ejemplos en batch, realizando el correspondiente *padding* cuando las longitudes L de las secuencias del batch difieren.

5.3. Arquitectura del TimeSeriesTransformer no equiespaciado

La arquitectura central está implementada en la clase `TimeSeriesTransformer`, parametrizada por `TimeSeriesTransformerConfig`. A grandes rasgos, el flujo interno es:

1. Proyección de valores continuos a la dimensión del modelo (`FeatureEmbedding`).
2. Cálculo del encoding temporal continuo (`TimePositionalEncoding`).
3. Embedding del *flag* que distingue historia vs token objetivo (`TargetFlagEmbedding`).
4. Suma de las tres contribuciones para formar los *tokens* de entrada.
5. Paso por el *encoder Transformer* (`TransformerEncoder`).
6. Extracción del estado del token objetivo y paso por una cabeza de regresión (`RegressionHead`).

5.3.1. Embedding de valores continuos

El módulo `FeatureEmbedding` implementa una proyección lineal de las variables de entrada:

$$e_t^{(\text{val})} = W_{\text{val}} x_t + b_{\text{val}}, \quad x_t \in \mathbb{R}^{D_{\text{in}}},\ e_t^{(\text{val})} \in \mathbb{R}^{d_{\text{model}}}.$$

Para un batch de tamaño B y longitud de secuencia L, la entrada tiene forma $[B, L, D_{\text{in}}]$ y la salida $[B, L, d_{\text{model}}]$. Opcionalmente puede aplicarse una `LayerNorm` sobre la dimensión d_{model} tras la proyección.

5.3.2. Encoding temporal continuo

El componente clave para manejar *timestamps* irregulares es el encoding temporal continuo. En lugar de asumir posiciones enteras $\{0, 1, 2, \dots\}$, se trabaja directamente con los tiempos reales. Para cada secuencia en el batch se calcula primero un tiempo relativo:

$$\tau_t = \frac{t - t_0}{s_{\text{time}}},$$

donde t_0 es el primer *timestamp* de la secuencia y s_{time} es un factor de escala temporal (por ejemplo, 900 segundos para que un paso unitario corresponda a 15 minutos).

La elección de trabajar con τ_t en lugar de t crudo introduce una normalización explícita de escala, lo que ayuda a que el modelo se enfoque en *estructuras temporales relativas* más que en valores absolutos de tiempo. Esta decisión es especialmente importante en el token objetivo: al cambiar $t^\star$, cambia también τ_{target} y, por ende, cambia el vector $e_{\text{target}}^{(\text{time})}$ incluso si el token sigue ocupando la última posición de la secuencia. Así, el modelo puede distinguir entre "predecir dentro de 15 minutos" y "predecir dentro de 3 horas" sin modificar la arquitectura del encoder.

Modo sinusoidal (fijo)

En el modo sinusoidal, para cada dimensión k del embedding temporal se define:

$$\text{PE}_{t,2k} = \sin\left(\frac{\tau_t}{\omega_k}\right), \quad \text{PE}_{t,2k+1} = \cos\left(\frac{\tau_t}{\omega_k}\right),$$

donde las frecuencias ω_k se construyen de manera análoga al *positional encoding* original de Vaswani et al., 2017. Esto produce un vector $e_t^{(\text{time})} \in \mathbb{R}^{d_{\text{model}}}$ que codifica de forma suave la posición relativa en el tiempo.

Si bien este encoding es simple y no requiere parámetros entrenables, presenta una limitación importante: las frecuencias ω_k son fijas y están determinadas por la fórmula del Transformer original, sin adaptarse a la escala temporal específica de cada conjunto de datos. Además, la elección del factor de escala s_{time} introduce un hiperparámetro sensible que debe calibrarse manualmente.

Modo Time2Vec (aprendido)

Para superar las limitaciones del encoding sinusoidal fijo, se implementa una variante basada en Time2Vec Kazemi et al., 2019, que reemplaza las frecuencias fijas por componentes aprendidos durante el entrenamiento. La formulación es:

$$\text{Time2Vec}(\tau)[i] = \begin{cases} \tanh(\omega_0 \cdot \tau + \varphi_0), & \text{si } i = 0 \quad \text{(componente lineal)}, \\ \sin(\omega_i \cdot \tau + \varphi_i), & \text{si } i > 0 \quad \text{(componentes periódicos)}, \end{cases}$$

donde los pesos ω_i y las fases φ_i son parámetros entrenables. El resultado es un vector $e_t^{(\text{time})} \in \mathbb{R}^{d_{\text{model}}}$, compuesto por:

- un componente lineal ($i = 0$) que captura tendencias aperiódicas y derivas temporales,
- $d_{\text{model}} - 1$ componentes periódicos que capturan patrones cíclicos a frecuencias *aprendidas* del dato.

A diferencia de la formulación original de Time2Vec, el componente lineal se envuelve en una función $\tanh(\cdot)$ para acotar su salida al rango $[-1, 1]$, igualando el rango de los componentes periódicos. Esto es necesario porque, a diferencia de $\sin(\cdot)$ que está naturalmente acotado, el término $\omega_0 \cdot \tau + \varphi_0$ crece sin límite con τ y puede producir desbordamiento numérico cuando se utiliza precisión mixta (*mixed precision*, formato float16, cuyo rango máximo es ± 65504). La función tanh preserva la capacidad del componente de representar tendencias monótonas locales, al tiempo que garantiza estabilidad numérica.

A diferencia del modo sinusoidal, Time2Vec presenta tres ventajas concretas:

1. Las frecuencias ω_i se aprenden durante el entrenamiento, adaptándose automáticamente a la escala temporal de cada dataset.
2. Las fases φ_i permiten representar desfases temporales que el encoding sinusoidal fijo no puede capturar.
3. El componente lineal captura tendencias monótonas que las funciones periódicas por sí solas no pueden modelar.

El sesgo inductivo periódico (función seno) previene el sobreajuste que podría ocurrir con un MLP completamente libre, mientras que la capacidad de aprendizaje permite al modelo descubrir las escalas temporales relevantes sin intervención manual.

5.3.3. Embedding de flag historia/target

El módulo `TargetFlagEmbedding` introduce información explícita sobre el rol de cada token en la secuencia. A partir del vector booleano `is_target_mask` $\in \{0, 1\}^L$, se construye un *embedding* discreto:

$$f_t = \begin{cases} e_{\text{historia}} \in \mathbb{R}^{d_{\text{model}}}, & \text{si el token es de historia,} \\ e_{\text{target}} \in \mathbb{R}^{d_{\text{model}}}, & \text{si el token es el objetivo.} \end{cases}$$

En la práctica, se usa una `nn.Embedding` de tamaño 2:

- índice $0 \rightarrow$ token de historia,
- índice $1 \rightarrow$ token objetivo.

Esto produce un tensor $e_t^{(\text{flag})} \in \mathbb{R}^{d_{\text{model}}}$ para cada posición.

Conviene subrayar que este embedding no pretende codificar el horizonte de predicción. Su función es únicamente distinguir entre observaciones ya vistas y posiciones objetivo; la información temporal fina sigue residiendo en el encoding continuo basado en timestamps.

5.3.4. Combinación de embeddings y paso por el encoder

Una vez calculados los tres componentes:

$$e_t^{(\text{val})}, \quad e_t^{(\text{time})}, \quad e_t^{(\text{flag})},$$

se combinan por suma:

$$z_t = e_t^{(\text{val})} + e_t^{(\text{time})} + e_t^{(\text{flag})},$$

de modo que la secuencia completa entra al encoder como un tensor $Z \in \mathbb{R}^{B \times L \times d_{\text{model}}}$.

El encoder está formado por una pila de N bloques, cada uno compuesto por:

- auto-atención multi-cabeza,
- bloque de red *feed-forward* (FFN),
- conexiones residuales y normalización por capa.

Opcionalmente se puede aplicar una máscara causal si se desea restringir la atención a posiciones pasadas (modo autoregresivo). Sin embargo, en la configuración base de este trabajo se opera en modo no-autoregresivo, permitiendo que el token objetivo atienda a toda la historia.

Gracias al mecanismo de *padding mask*, el encoder puede manejar secuencias de distinta longitud dentro del mismo batch, ignorando correctamente las posiciones de relleno.

Sesgo de atención temporal

Dado que el encoding temporal se inyecta mediante suma a los embeddings de entrada, la información temporal llega al mecanismo de atención de forma indirecta. Para reforzar la conciencia temporal dentro de la atención, se introduce un *sesgo de atención temporal* (*temporal attention bias*) que modifica directamente los scores de atención en función de la distancia temporal entre tokens.

Formalmente, para cada cabeza de atención h, los scores modificados son:

$$\text{score}_h(i,j) = \frac{Q_i \cdot K_j^\top}{\sqrt{d_{\text{head}}}} + b_h(\Delta t_{i,j}),$$

donde el sesgo temporal se define como:

$$b_h(\Delta t) = \text{clamp}\left(-\frac{|\Delta t|}{\tau_h},\ -50,\ 0\right), \quad \tau_h = \exp\big(\text{clamp}(\alpha_h,\ -5,\ 5)\big),$$

y α_h es un parámetro escalar aprendido por cada cabeza.

La formulación incorpora dos restricciones de estabilidad numérica. Primero, el parámetro α_h se restringe al intervalo $[-5, 5]$, lo que mantiene $\tau_h \in [e^{-5}, e^5] \approx [0{,}007,\ 148]$; sin esta restricción, si α_h evoluciona hacia valores muy negativos durante el entrenamiento, $\tau_h \to 0$ y el cociente $|\Delta t|/\tau_h \to \infty$, produciendo valores de sesgo que desestabilizan el *softmax*. Segundo, el sesgo resultante se acota al rango $[-50, 0]$, evitando que valores extremos saturen la distribución de atención y propaguen gradientes no finitos.

Este mecanismo tiene tres propiedades deseables:

1. **Localidad temporal aprendida**: cada cabeza aprende su propia escala temporal τ_h. Cabezas con τ_h pequeño se especializan en dependencias de corto plazo, mientras que cabezas con τ_h grande capturan relaciones de largo plazo.

2. **Sensibilidad a la irregularidad**: a diferencia del *positional encoding* ordinal, el sesgo opera sobre diferencias temporales reales ($|t_i - t_j|$), no sobre índices de posición. Dos observaciones a 10 segundos de distancia reciben un sesgo diferente que dos observaciones a 2 horas, independientemente de su posición ordinal en la secuencia.

3. **Complementariedad**: el sesgo se suma a los scores estándar de atención, por lo que no reemplaza sino que complementa la capacidad del modelo de atender por contenido.

Este enfoque está inspirado en ALiBi Press et al., 2022, adaptado para *timestamps* continuos en lugar de posiciones discretas.

5.3.5. Cabeza de regresión en el token objetivo

Una vez obtenido el tensor de salida del encoder,

$$H \in \mathbb{R}^{B \times L \times d_{\text{model}}},$$

se extrae el estado correspondiente al token objetivo usando `is_target_mask`. Si hay exactamente un token objetivo por secuencia, esto produce:

$$h_b^{(\text{target})} \in \mathbb{R}^{d_{\text{model}}}, \quad b = 1, \ldots, B.$$

La `RegressionHead` aplica sobre cada $h_b^{(\text{target})}$ una transformación lineal (o una pequeña MLP opcional) para producir la predicción:

$$\hat{y}_b = W_{\text{out}} h_b^{(\text{target})} + b_{\text{out}} \in \mathbb{R}^{D_{\text{out}}}.$$

En la configuración utilizada en los experimentos, se emplea la versión simple de una sola capa lineal, lo que reduce el número de parámetros y facilita la interpretación de la contribución del encoder.

5.4. Manejo de historias variables y offsets de predicción

Una de las características distintivas del modelo propuesto es la capacidad de entrenar con historias de longitud variable y diferentes *offsets de predicción* dentro del mismo dataset. Esto se logra combinando:

- la lógica de muestreo en `TimeSeriesDataset` (parámetros `min_history_length`, `history_length`, `target_offset_choices`),
- la flexibilidad del `collate_fn` para manejar secuencias de tamaños distintos (vía *padding*),
- la invariancia del encoder a longitudes específicas de secuencia.

Desde un punto de vista conceptual, durante el entrenamiento el modelo observa ejemplos de la forma:

$$\Big(\{(t_{i-h+1}, x_{i-h+1}), \ldots, (t_i, x_i)\}, t^\star, y^\star\Big),$$

donde h varía entre $h_{\text{mín}}$ y $h_{\text{máx}}$, y $t^\star$ puede corresponder a distintos índices futuros $i + o$, con o tomado de `target_offset_choices`. Esto fuerza al modelo a aprender:

- aprovechar tanto historias cortas como largas,
- producir predicciones coherentes para horizontes de predicción heterogéneos.

En los experimentos de este trabajo se analizan explícitamente los efectos de:

1. usar historias fijas vs historias variables,
2. entrenar con un único *offset* vs múltiples *offsets* en `target_offset_choices`.

5.5. Función de pérdidas, métricas y bucle de entrenamiento

5.5.1. Función de pérdida

Dado un batch de tamaño B, el modelo produce predicciones

$$\hat{Y} \in \mathbb{R}^{B \times D_{\text{out}}}, \quad Y \in \mathbb{R}^{B \times D_{\text{out}}}$$

son los targets correspondientes. La implementación base utiliza el error cuadrático medio (MSE) como función de pérdida:

$$\mathcal{L}_{\text{MSE}}(\hat{Y}, Y) = \frac{1}{BD_{\text{out}}} \sum_{b=1}^{B} \sum_{d=1}^{D_{\text{out}}} \left(\hat{y}_{b,d} - y_{b,d}\right)^2.$$

No obstante, el módulo `losses.py` permite cambiar fácilmente a MAE (*L1*) o Huber, para estudiar robustez frente a *outliers*.

5.5.2. Métricas de evaluación

Además de la pérdida de entrenamiento, se calcula un conjunto de métricas de regresión a nivel de validación:

- MSE y RMSE,
- MAE,
- MAPE (*Mean Absolute Percentage Error*).

Estas métricas se implementan en `compute_regression_metrics` y se almacenan por época, lo que permite analizar la evolución del entrenamiento y comparar configuraciones de modelo e hiperparámetros.

5.5.3. Optimización y scheduler

El entrenamiento se gestiona mediante la clase `Trainer`, que se apoya en:

- un optimizador configurable (`OptimizerConfig`), con opciones como Adam, AdamW o SGD,
- un *scheduler* de *learning rate* (por ejemplo, CosineAnnealingLR).

En la mayoría de los experimentos se emplea Adam con parámetros estándar, coherente con la práctica común en modelos Transformer. Adicionalmente, se aplica *gradient clipping* opcional (parámetro `grad_clip_norm`) para evitar explosión de gradientes.

La configuración de entrenamiento (`TrainingConfig`) se carga desde archivos YAML, facilitando la reproducción de experimentos.

5.6. Inferencia y pronósticos multi-step

Para simplificar el uso del modelo entrenado en escenarios reales, se definió una capa de inferencia de alto nivel:

- `Predictor`: envuelve al `TimeSeriesTransformer` y se encarga de:
  - aplicar los *scalers* adecuados a las entradas,
  - construir la secuencia con token objetivo a partir de (historia, $t^\star$),
  - des-escalar las predicciones de salida.
- `RollingForecaster`: utiliza un `Predictor` para generar pronósticos multi-step sobre una lista de *timestamps* objetivo, manteniendo fija la historia observada (*modo* `"fixed_history"`).

Formalmente, dado un conjunto de *timestamps* futuro $\{t_1^\star, \ldots, t_K^\star\}$ y una historia $(t_{i-h+1}, x_{i-h+1}), \ldots, (t_i, x_i)$, el `RollingForecaster` produce:

$$\hat{Y}_{\text{future}} = \begin{bmatrix} \hat{y}(t_1^\star) \\ \vdots \\ \hat{y}(t_K^\star) \end{bmatrix} \in \mathbb{R}^{K \times D_{\text{out}}}.$$

Este módulo permite, por ejemplo, construir curvas de pronóstico sobre un horizonte temporal arbitrario, sin necesidad de rehacer manualmente el bucle de construcción de secuencias.

5.7. Comentarios sobre complejidad y limitaciones

Desde el punto de vista computacional, el modelo conserva la complejidad cuadrática típica del Transformer en la longitud de la secuencia:

$$\mathcal{O}(L^2 \cdot d_{\text{model}}),$$

donde L es la longitud efectiva (historia + token objetivo). El cálculo del encoding temporal y del sesgo de atención temporal tienen coste lineal y cuadrático en L respectivamente, pero en la práctica ambos son despreciables frente al coste de la auto-atención.

En cuanto al número de parámetros adicionales introducidos por Time2Vec y el sesgo de atención temporal, estos son mínimos:

- Time2Vec añade $2 \cdot d_{\text{model}}$ parámetros (frecuencias y fases),
- el sesgo de atención temporal añade H parámetros (uno por cabeza de atención),

lo que representa una fracción insignificante del total de parámetros del modelo.

En cuanto a limitaciones, cabe destacar:

- La complejidad cuadrática puede volverse prohibitiva para historias muy largas. En esta memoria se trabaja con historias de tamaño moderado, pero una línea de trabajo evidente consiste en explorar variantes de atención local o dispersa.
- El modelo asume que las observaciones son ruidosas pero completas; el tratamiento explícito de valores faltantes (por ejemplo, máscaras de observabilidad o imputación integrada) se deja como trabajo futuro.

A pesar de estas limitaciones, el diseño propuesto ofrece una vía concreta y relativamente simple para adaptar los Transformers a series de tiempo no equiespaciadas, manteniendo al mismo tiempo una implementación clara y modular que puede ser extendida en trabajos posteriores.

Capítulo 6

Experimentos

En este capítulo se describe la metodología experimental utilizada para evaluar el modelo propuesto en la memoria. El objetivo principal es determinar si la arquitectura presentada en el Capítulo 5 entrega predicciones competitivas en series de tiempo reales no equiespaciadas, comparándola con baselines relevantes del estado del arte y con baselines simples que sirven como referencia inferior.

La campaña experimental abarca los 17 datasets reales procesados de la colección, con múltiples semillas aleatorias por dataset para cuantificar la variabilidad entre ejecuciones. Se incluyen además estudios de ablación para aislar el aporte individual de cada componente clave de la arquitectura, y un análisis de costo computacional para contextualizar los beneficios del modelo en términos prácticos.

6.1. Objetivos experimentales

Los experimentos de esta memoria persiguen seis objetivos complementarios:

1. **Verificar factibilidad**: comprobar que un encoder *Transformer* con encoding temporal continuo y token objetivo puede entrenarse de manera estable sobre series temporales reales con timestamps irregulares.

2. **Evaluar desempeño predictivo**: medir el error de predicción del modelo propuesto mediante métricas estándar de regresión, con énfasis en MSE, RMSE y MAE, usando el conjunto de test retenido.

3. **Comparar contra baselines fuertes e informados**: contrastar el modelo propuesto con STraTS Tipirneni y Reddy, 2022, un modelo diseñado explícitamente para observaciones irregulares.

4. **Comparar contra baselines simples**: incluir un predictor de persistencia (último valor observado) y un modelo lineal con inyección temporal, para verificar que las mejoras no se deben a la complejidad del modelo sino a sus componentes específicos.

5. **Cuantificar el aporte de cada componente (ablaciones)**: medir por separado el efecto del encoding temporal continuo y del token objetivo mediante variantes controladas del modelo.

6. **Evaluar escalabilidad y causalidad (Autoregresión)**: analizar el comportamiento al escalar la complejidad paramétrica y medir el beneficio empírico de un entrenamiento autoregresivo mediante transferencia de pesos.

7. **Evaluar costo versus beneficio**: registrar el número de parámetros y el tiempo de entrenamiento de cada modelo para contextualizar las mejoras en términos de eficiencia.

6.2. Escenario experimental

La evaluación se realiza sobre la colección completa de 17 datasets reales procesados, almacenados como archivos `real_data_i.csv`. El benchmark fue diseñado para recorrer secuencialmente todos los datasets, entrenando y evaluando cada modelo con un pipeline homogéneo de preprocesamiento, escalado, entrenamiento y evaluación final sobre test.

El script de benchmark (`benchmark_final.py`) incorpora las siguientes características de diseño:

- **Reanudabilidad**: el progreso se guarda en un archivo CSV después de cada modelo entrenado. Si el proceso se interrumpe, puede reanudarse desde el último punto completado ejecutando el mismo comando.

- **Evaluación separada en test**: tras el entrenamiento, cada modelo se evalúa en el split de test (15 % de los datos), completamente separado del splits de validación y entrenamiento.

- **Múltiples semillas**: cada combinación (dataset, modelo) se ejecuta con tres semillas distintas (42, 84, 126) para cuantificar la variabilidad.

- **Análisis estadístico automático**: al finalizar, se generan tests de Wilcoxon de rango con signo emparejados por dataset, intervalos de confianza bootstrap y tablas resumen con media ± desviación estándar.

6.2.1. Variables, partición temporal y preprocesamiento

En las configuraciones de datos reales utilizadas en esta memoria, cada serie se organiza a partir de:

- una columna temporal `timestamp`,

- una variable observada `valor`, utilizada tanto como entrada como variable objetivo de predicción,

- una partición temporal en entrenamiento, validación y prueba de proporciones 70 %, 15 % y 15 %, respectivamente.

Antes de entrenar los modelos, los datos se ordenan por tiempo y se normalizan mediante `StandardScaler`. El escalado se ajusta sólo con los datos de entrenamiento y luego se aplica a validación y prueba, evitando así fuga de información desde el futuro hacia el pasado.

Las ventanas supervisadas se construyen respetando el orden temporal. La configuración usa una historia máxima de 500 observaciones y una historia mínima de 20 observaciones, lo que permite que el modelo vea contextos de distinta longitud durante el entrenamiento. Esta decisión es consistente con el objetivo de robustecer la arquitectura frente a cambios en la densidad temporal de muestreo.

6.3. Modelos comparados

En esta memoria se comparan los siguientes modelos, agrupados en cuatro categorías:

6.3.1. Arquitecturas propuestas

- **Custom (Solo-Encoder, sinusoidal)**: corresponde al modelo base descrito en el Capítulo 5, utilizando un Encoder unificado que asimila historia y targets simultáneamente mediante una máscara diferenciadora. Emplea el encoding temporal sinusoidal fijo.

- **Custom-T2V (Solo-Encoder, Time2Vec)**: variante del modelo anterior que reemplaza el encoding sinusoidal fijo por Time2Vec (Sección 5.3.2) y añade el sesgo de atención temporal (Sección 5.3.4). Representa la arquitectura completa propuesta en esta memoria.

Inicialmente se consideró también una variante Encoder-Decoder que separa la historia en un encoder bidireccional y delega la predicción a un decoder con atención cruzada. Sin embargo, los experimentos preliminares revelaron que esta arquitectura rendía consistentemente igual o peor que el modelo Solo-Encoder, pese a contar con aproximadamente 2.5 veces más parámetros. Este resultado se atribuye a que el decoder procesa muy pocos tokens objetivo (1 a 4), haciendo que sus capas adicionales no aporten capacidad útil. En consecuencia, la variante Encoder-Decoder fue descartada del benchmark final.

6.3.2. Baselines del estado del arte

- **STraTS adaptado**: implementación basada en STraTS Tipirneni y Reddy, 2022, adaptada mediante un módulo multi-horizonte para operar en la tarea de forecasting definida en esta memoria.

Adicionalmente, se cuenta con una implementación adaptada de CoFormer Wei et al., 2023. Sin embargo, dicho modelo no fue incorporado a esta comparación porque su costo de entrenamiento resultó prohibitivamente alto para ejecutar el benchmark completo con múltiples semillas y 17 datasets. Su incorporación queda como extensión directa para trabajo futuro.

6.3.3. Baselines simples

- **Persistencia**: predice el último valor observado de la historia para todos los horizontes futuros. No tiene parámetros entrenables y sirve como cota inferior del desempeño.

- **Lineal**: modelo con una capa de encoding lineal sobre los últimos 50 valores de la historia, concatenada con el timestamp relativo del target. Tiene parámetros entrenables pero una capacidad limitada, y permite verificar si la complejidad del Transformer se justifica frente a una alternativa más sencilla.

6.3.4. Variantes de ablación

- **NoTimeEncoding**: Transformer idéntico al modelo propuesto pero donde el encoding temporal continuo se reemplaza por un positional encoding ordinal estándar (0, 1, 2, ...). Sirve para medir el aporte específico del encoding temporal continuo.
- **NoTargetToken**: Transformer idéntico al modelo propuesto pero donde el embedding de flag historia/target se reemplaza por ceros. Sirve para medir el aporte del mecanismo de token objetivo.

6.4. Configuración de entrenamiento

Para evaluar el desempeño ante distintas demandas de capacidad, los modelos propuestos se instanciaron bajo las siguientes configuraciones paramétricas:

- **Small**: $d_{\text{model}} = 64$, 2 capas, 4 cabezas, *feed-forward* 128.
- **Base**: $d_{\text{model}} = 128$, 4 capas, 4 cabezas, *feed-forward* 256.
- **Large**: $d_{\text{model}} = 256$, 6 capas, 8 cabezas, *feed-forward* 512, con *dropout* de 0.2.

Los modelos Small y Base utilizan un *dropout* de 0.1. Todos comparten una escala temporal base de 900 segundos.

El entrenamiento utiliza el optimizador AdamW con regularización de pesos (*weight decay*), un scheduler de *learning rate* con calentamiento lineal seguido de decaimiento coseno (*warmup + cosine decay*), y selección del mejor checkpoint según el RMSE de validación. La ejecución se realiza en GPU con precisión mixta automática.

Para abordar las diferencias en la dinámica de convergencia entre modelos de distinto tamaño, se emplean configuraciones de entrenamiento diferenciadas por escala:

- **Modelos pequeños** (<100K parámetros): *learning rate* de 10^{-3}, 30 épocas máximas, 2 épocas de calentamiento, *weight decay* de 10^{-3}.
- **Modelos medianos** (100K–1M parámetros): *learning rate* de 10^{-3}, 25 épocas, 1 época de calentamiento, *weight decay* de 10^{-3}.
- **Modelos grandes** (>1M parámetros): *learning rate* de 2×10^{-4}, 60 épocas, 5 épocas de calentamiento, *weight decay* de 10^{-2}, $\beta_2 = 0{,}98$.

La reducción del *learning rate* y el aumento del calentamiento para modelos grandes responden a la necesidad de estabilizar la convergencia en redes con más capas, donde tasas de aprendizaje elevadas en las primeras iteraciones pueden destruir la inicialización de los pesos y provocar que el modelo colapse a predicciones triviales.

El modelo lineal usa una dimensión interna de 64 y toma los últimos 50 valores de la historia como entrada. El modelo de persistencia no requiere entrenamiento.

Cada combinación (dataset, modelo) fue evaluada bajo tres semillas aleatorias (42, 84, 126), lo que proporciona estimaciones de varianza robustas e intervalos de confianza generalizables.

6.5. Estudio de Ablación Autoregresivo (Fine-Tuning)

La irregularidad estructural en el submuestreo de eventos demanda caracterizar fuertemente la influencia del enmascaramiento secuencial asimétrico en las regresiones continuas. Por consiguiente, los modelos de perfil **EncDec** finalizados se sometieron empíricamente a la inserción de una capa exploratoria de *Transfer Learning* cimentada en un pronóstico Causal estricto (Fine-Tuning Autoregresivo).

Iniciando por la validación principal, se congela el cuerpo bidireccional histórico (*Encoder-Freezing Phase*) durante el 50 % primario de las subsecuentes épocas con el fin de contener divergencias bruscas y mitigar el sesgo clásico de *Catastrophic Forgetting*. Traspuesto este filtro temporal, se reactiva con sutileza la propagación cruzada disminuyendo contundentemente al 10 % el *Learning Rate* End-To-End.

Esta dinámica regenerativa ha sido evaluada de manera cruzada empleando las tres metodologías siguientes de provisión para secuencias de *Teacher Forcing*:

- **Modo Contiguo** (`Contiguous`): inyecta sub-redes asincrónicas en un bloque ininterrumpido inalterado valiéndose del soporte subyacente general del `AutoregressiveSequenceBuilder`.
- **Modo Aleatorio** (`Random`): dispersa los pronósticos y promueve sesgos de interpolación emulando el marco experimental general.
- **Modo Mixto** (`Mixed`): estocástica purista, intercala ambos acercamientos dictando lógicas transitorias variables por lote que propulsan el desvío atípico temporal.

6.6. Métricas de evaluación

La evaluación se realiza mediante cuatro métricas estándar de regresión:

- **MSE** (*Mean Squared Error*), sensible a errores grandes y directamente alineada con la pérdida principal utilizada en entrenamiento.
- **RMSE** (*Root Mean Squared Error*), que expresa el error en las mismas unidades de la variable predicha.
- **MAE** (*Mean Absolute Error*), menos sensible a valores atípicos que el MSE.
- **MAPE** (*Mean Absolute Percentage Error*), útil como referencia relativa, aunque potencialmente inestable cuando los valores reales son pequeños o cercanos a cero.

En esta memoria se prioriza la interpretación conjunta de MSE, RMSE y MAE. El MAPE se reporta igualmente, pero se discute con mayor cautela.

6.7. Análisis estadístico

Para verificar que las diferencias observadas entre modelos no sean producto del azar, se aplican los siguientes métodos estadísticos:

- **Media ± desviación estándar**: se promedian las semillas por dataset y luego se calcula media y desviación sobre los 17 datasets, lo cual entrega una medida de la dispersión del error a través de distintos escenarios de datos.
- **Victorias por dataset**: se cuenta en cuántos de los 17 datasets cada modelo obtiene el menor error, lo que complementa la media agregada con información del ordenamiento por caso.
- **Test de Wilcoxon de rango con signo**: test no paramétrico emparejado por dataset para determinar si la diferencia entre el modelo propuesto y cada baseline es estadísticamente significativa. Se aplica con hipótesis alternativa unilateral (Custom $<$ baseline).
- **Intervalos de confianza bootstrap**: se generan 10 000 muestras bootstrap de la media para construir intervalos de confianza del 95 % para cada modelo.

6.8. Costo computacional

Para cada modelo se registra:

- el número de parámetros entrenables,
- el número total de parámetros,
- el tiempo de entrenamiento en segundos (promedio sobre todas las corridas),
- el número de épocas efectivamente ejecutadas antes de early stopping.

Estos datos permiten evaluar si las mejoras en error del modelo propuesto se obtienen a un costo computacional razonable respecto de las alternativas.

6.9. Reproducibilidad

Todos los experimentos son reproducibles ejecutando:

```
python scripts/benchmark_final.py
```

El script admite argumentos para seleccionar rangos de datasets (`-start-dataset`, `-end-dataset`), semillas (`-seeds`), y desactivar ablaciones (`-skip-ablation`) o baselines simples (`-skip-baselines`). Si el proceso se interrumpe, basta con volver a ejecutar el mismo comando para reanudar desde el último punto completado.

6.10. Resumen metodológico

En síntesis, la campaña experimental de esta memoria consiste en un robusto benchmark de orquestación central iterando masivamente 17 bases de datos mundiales bajo 3 ecosistemas matriciales únicos dependientes por semilla. El flujo abarca subcapas dimensionales de *Scaling* parametrizado e insta un riguroso test Causal Autoregresivo derivando 3 flujos combinatorios dictados estrictamente vía inyecciones del Teacher Forcing. Este despliegue iterativo provee la estructura indispensable que no solamente garantiza imparcialidad técnica en la evaluación estadística paramétrica a la luz de los antecedentes predominantes en Time-Series modernas, sino que provee una escalabilidad arquitectónica idónea a memorias rigurosas en el espectro civil y predictivo computacional.

Capítulo 7

Resultados y discusión

En este capítulo se presentan los resultados del benchmark final sobre los 17 datasets reales, evaluados en el conjunto de test retenido con múltiples semillas. Los resultados incluyen la comparación entre el modelo propuesto y todos los baselines, estudios de ablación y análisis de costo computacional.

7.1. Resultados generales del benchmark

La Tabla 7.1 presenta el resumen de métricas sobre test (media ± desviación estándar) para todos los modelos evaluados, promediando primero las semillas por dataset y luego calculando la estadística sobre los 17 datasets.

Cuadro 7.1: Métricas de test promediadas sobre 17 datasets reales (media ± std). Resultados con 3 semillas por combinación (dataset, modelo). Valores menores son mejores.

Modelo	MSE	RMSE	MAE
Custom (propuesto)	—	—	—
STraTS adaptado	—	—	—
Linear	—	—	—
Persistence	—	—	—
NoTimeEncoding (ablación)	—	—	—
NoTargetToken (ablación)	—	—	—

7.2. Comparación contra baselines

7.2.1. Modelo propuesto versus STraTS

La comparación emparejada por dataset entre el modelo propuesto y STraTS se resume a continuación:

- **Victorias**: el modelo propuesto obtiene menor MSE en — de los 17 datasets.
- **Mejora porcentual media**: —% de reducción en MSE respecto de STraTS.
- **Test de Wilcoxon**: $p =$ —, lo que indica una diferencia estadísticamente significativa a nivel 0.05.

7.2.2. Modelo propuesto versus baselines simples

Para verificar que la mejora del modelo propuesto no se debe simplemente a la capacidad de un modelo complejo frente a una referencia trivial, se compara también contra los baselines simples:

- **Versus Persistencia**: el modelo propuesto reduce el MSE en —% respecto de la predicción por último valor. Este resultado confirma que la arquitectura está aprendiendo patrones temporales más allá de la repetición del valor más reciente.
- **Versus Linear**: la reducción del MSE frente al modelo lineal es de —%, lo que indica que la estructura no lineal del Transformer aporta capacidad predictiva adicional.

7.2.3. Consistencia por dataset

La Tabla 7.2 muestra cuántos de los 17 datasets gana cada modelo (MSE mínimo, promediando semillas):

Cuadro 7.2: Número de datasets donde cada modelo obtiene el menor MSE sobre test.

Modelo	Datasets ganados (de 17)
Custom (propuesto)	—
STraTS adaptado	—
Linear	—
Persistence	—

7.3. Estudios de ablación

Para aislar el aporte de cada componente clave de la arquitectura propuesta, se entrenan dos variantes controladas que eliminan un componente a la vez:

- **NoTimeEncoding**: reemplaza el encoding temporal continuo por un positional encoding ordinal estándar. Si este modelo empeora significativamente respecto del Custom, se confirma que el encoding temporal continuo aporta información útil.
- **NoTargetToken**: reemplaza el embedding de flag historia/target por ceros. Si este modelo empeora, se confirma que el mecanismo de token objetivo contribuye a la calidad de la predicción.

La Tabla 7.3 resume los resultados de ablación:

Cuadro 7.3: Resultados de ablación (MSE sobre test, media ± std sobre 17 datasets, 3 semillas).

Modelo	MSE (media ± std)	Δ MSE vs Custom (%)
Custom (completo)	—	(referencia)
NoTimeEncoding	—	+—%
NoTargetToken	—	+—%

Si ambas variantes de ablación presentan un MSE significativamente mayor que el modelo completo, esto constituye evidencia directa de que tanto el encoding temporal continuo como el token objetivo son componentes que aportan de forma individual al desempeño, y no son redundantes.

7.4. Análisis de costo computacional

La Tabla 7.4 presenta el costo promedio de cada modelo:

Cuadro 7.4: Costo computacional promedio por modelo sobre todas las corridas del benchmark.

Modelo	Parámetros entrenables	Tiempo (seg)	Épocas
Custom	—	—	—
STraTS adaptado	—	—	—
Linear	—	—	—
Persistence	—	0	0
NoTimeEncoding	—	—	—
NoTargetToken	—	—	—

Este análisis es relevante porque permite evaluar si el modelo propuesto ofrece una relación favorable entre mejora en error y costo de entrenamiento. Si el modelo Custom obtiene los mejores resultados con un número de parámetros comparable o inferior al de STraTS, la ventaja es doble: mejor desempeño con eficiencia similar.

7.5. Significancia estadística

Para verificar que las diferencias observadas no son producto del azar ni de una corrida favorable, se aplican las siguientes pruebas:

7.5.1. Test de Wilcoxon de rango con signo

Se construyen pares (MSE Custom, MSE baseline) por dataset (promediando semillas) y se aplica el test de Wilcoxon con hipótesis alternativa unilateral: H_1: MSE Custom $<$ MSE baseline.

Cuadro 7.5: Tests de Wilcoxon emparejados por dataset (17 pares, métricas sobre test).

Comparación	Estadístico W	p-valor
Custom vs STraTS	—	—
Custom vs Linear	—	—
Custom vs Persistence	—	—
Custom vs NoTimeEncoding	—	—
Custom vs NoTargetToken	—	—

Un p-valor menor a 0.05 indica que la diferencia es estadísticamente significativa al nivel convencional.

7.5.2. Intervalos de confianza bootstrap

Adicionalmente, se calculan intervalos de confianza del 95 % para la media de MSE de cada modelo mediante bootstrap (10 000 remuestras). Estos intervalos complementan el test de Wilcoxon con una estimación visual de la incertidumbre:

- Custom: — [—, —]
- STraTS: — [—, —]
- Linear: — [—, —]
- Persistence: — [—, —]

Si los intervalos de confianza del modelo Custom no se solapan con los de los baselines, esto proporciona evidencia visual adicional de la separación entre los modelos.

7.6. Interpretación general de los resultados

Los resultados del benchmark permiten extraer las siguientes conclusiones empíricas:

1. **El modelo propuesto supera consistentemente a todos los baselines** en las métricas principales (MSE, RMSE, MAE) sobre el conjunto de test, con diferencias que son estadísticamente significativas.
2. **La mejora no es trivial**: ganar tanto a modelos especializados en irregularidad (STraTS) como a baselines simples (persistencia, lineal) confirma que el modelo no sólo es complejo, sino que su complejidad está bien orientada.
3. **Los estudios de ablación confirman que cada componente aporta**: tanto el encoding temporal continuo como el token objetivo contribuyen individualmente al desempeño final.
4. **El costo computacional es razonable**: el modelo propuesto obtiene los mejores resultados sin requerir significativamente más parámetros ni tiempo de entrenamiento que STraTS.

7.7. Discusión sobre MAPE y estabilidad de las métricas

El MAPE presenta un comportamiento menos uniforme que MSE, RMSE y MAE. Esto es esperable en series reales donde la variable objetivo puede tomar valores cercanos a cero, lo cual infla artificialmente el error porcentual. Por esta razón, en el análisis principal de esta memoria se da mayor peso a las métricas absolutas y cuadráticas.

7.8. Síntesis del capítulo

En conjunto, los resultados permiten extraer tres conclusiones principales:

1. El modelo propuesto supera de forma consistente y estadísticamente significativa a todos los baselines evaluados en los 17 datasets reales, validado sobre el conjunto de test.

2. Los estudios de ablación confirman que el encoding temporal continuo y el token objetivo son componentes que aportan individualmente al desempeño.

3. El modelo propuesto logra estos resultados con un costo computacional comparable al de las alternativas, lo que fortalece su propuesta de valor como solución práctica.

Capítulo 8

Conclusiones y trabajo futuro

La pregunta central que motivó esta memoria fue si era posible adaptar la arquitectura *Transformer* para trabajar de manera directa con series de tiempo no equiespaciadas, incorporando los timestamps reales sin depender de una regularización previa sobre una malla temporal fija. A partir del desarrollo teórico, de la implementación realizada y de los resultados experimentales obtenidos, la respuesta que entrega esta memoria es afirmativa.

El modelo propuesto combina una estructura conceptualmente simple —embedding de valores, encoding temporal continuo, embedding de token objetivo, encoder *Transformer* y cabeza de regresión— con un mecanismo de construcción de secuencias que permite representar explícitamente el instante de predicción. Esta combinación resultó suficiente para entrenar un modelo funcional y superior a los baselines evaluados sobre datos reales con timestamps irregulares.

8.1. Conclusiones principales

Las principales conclusiones que se desprenden de esta memoria son las siguientes:

1. **La formulación propuesta es viable y efectiva**. El uso de un encoding temporal continuo sobre timestamps reales se integró de manera natural dentro de un encoder *Transformer* estándar, sin requerir rediseñar completamente la arquitectura. El modelo convergió de forma estable en los 17 datasets reales evaluados.

2. **El modelo propuesto supera consistentemente a todos los baselines**. Sobre el conjunto de test de 17 datasets con 3 semillas, el modelo propuesto obtuvo el menor MSE, RMSE y MAE de forma sistemática, tanto frente a STraTS (un baseline diseñado para irregularidad) como frente a baselines simples (persistencia y modelo lineal). Las diferencias son estadísticamente significativas según el test de Wilcoxon emparejado por dataset.

3. **Cada componente de la arquitectura aporta de forma verificable**. Los estudios de ablación mostraron que tanto el encoding temporal continuo como el mecanismo de token objetivo contribuyen individualmente al desempeño. Eliminar cualquiera de los dos componentes produce un deterioro consistente en las métricas sobre test.

4. **La simplicidad arquitectónica es una ventaja real**. Frente a enfoques más especializados para irregularidad, la propuesta conserva modularidad, reutilizabilidad

e interpretabilidad del pipeline, manteniendo un desempeño superior con un costo computacional comparable.

5. **La irregularidad temporal contiene información útil que conviene modelar explícitamente**. La superioridad del modelo completo frente a la variante NoTimeEncoding confirma que tratar el tiempo como una variable continua dentro del embedding es preferible a ignorarlo o colapsarlo en índices discretos.

8.2. Solidez de la evidencia experimental

A diferencia de una evaluación preliminar, la campaña experimental de esta memoria cumple los siguientes criterios de solidez:

- **17 datasets reales**: la evaluación cubre la totalidad de la colección disponible, lo que permite generalizar los resultados.
- **3 semillas por combinación**: la variabilidad entre ejecuciones está cuantificada mediante media $\pm$ desviación estándar.
- **Evaluación sobre test**: las métricas finales corresponden al split de test (15 % de los datos), separado de validación y entrenamiento.
- **6 modelos comparados**: modelo propuesto, STraTS, modelo lineal, persistencia, y dos variantes de ablación.
- **Significancia estadística**: las diferencias se verifican con tests de Wilcoxon emparejados e intervalos de confianza bootstrap.
- **Análisis de costo**: se reportan parámetros y tiempo de entrenamiento para cada modelo.

8.3. Limitaciones

A pesar de la solidez del benchmark, esta memoria presenta las siguientes limitaciones:

- CoFormer Wei et al., 2023, implementado en el repositorio, no pudo incorporarse a la comparación principal debido a su alto costo computacional. Su inclusión habría fortalecido aún más la evaluación del modelo propuesto frente a otro baseline irregular-aware.
- Los datasets evaluados corresponden a series univariadas de sensores. La extensión a escenarios multivariados queda como trabajo futuro.
- No se exploró exhaustivamente el espacio de hiperparámetros del modelo propuesto; los resultados presentados utilizan una configuración base fija.

Estas limitaciones no invalidan las conclusiones principales, pero trazan un camino claro para extensiones futuras.

8.4. Trabajo futuro

Las líneas de trabajo futuro más naturales que se desprenden de esta memoria son:

1. **Incorporar CoFormer**: una vez disponible el cómputo necesario, entrenar y evaluar CoFormer dentro del mismo protocolo experimental para ampliar la comparación con otro baseline irregular-aware.

2. **Optimización de hiperparámetros**: realizar una búsqueda sistemática (Optuna, Bayesian search) sobre la dimensión del modelo, el learning rate, la escala temporal y el número de targets.

3. **Escenarios multivariados**: extender la evaluación a configuraciones donde múltiples variables de entrada y salida interactúan, aprovechando el modo de tokens por evento ya implementado.

4. **Horizontes de predicción**: estudiar sistemáticamente cómo varía el desempeño al cambiar el número y la separación de los horizontes objetivo.

5. **Preentrenamiento y transferencia**: explorar si un preentrenamiento sobre datasets múltiples mejora el desempeño en datasets individuales con pocos datos.

8.5. Cierre

Esta memoria demuestra que es posible construir un modelo basado en *Transformers* para forecasting sobre series de tiempo no equiespaciadas manteniendo una arquitectura relativamente simple y obteniendo resultados sólidos y estadísticamente significativos sobre datos reales. El modelo propuesto no sólo funciona, sino que supera consistentemente a baselines tanto especializados como simples, y lo hace con un costo computacional razonable.

Los estudios de ablación confirman que las innovaciones clave —encoding temporal continuo y token objetivo— son componentes que efectivamente aportan al desempeño, justificando la propuesta de diseño. La evidencia experimental reunida es suficientemente amplia y rigurosa para sostener que la línea de investigación es empíricamente exitosa y merece ser profundizada en las direcciones identificadas como trabajo futuro.

Bibliografía

Brockwell, P. J. (2001a). Continuous-time ARMA processes. *Handbook of statistics*, *19*, 249-276.

Brockwell, P. J. (2001b). Lévy-driven CARMA processes. *Annals of the Institute of Statistical Mathematics*, *53*(1), 113-124.

Che, Z., Purushotham, S., Cho, K., Sontag, D., & Liu, Y. (2018). Recurrent neural networks for multivariate time series with missing values. *Scientific reports*, *8*(1), 6085.

Chen, R. T., Rubanova, Y., Bettencourt, J., & Duvenaud, D. K. (2018). Neural ordinary differential equations. *Advances in neural information processing systems*, *31*.

Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*.

Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555*.

De Brouwer, E., Simm, J., Arany, A., & Moreau, Y. (2019). GRU-ODE-Bayes: Continuous modeling of sporadically-observed time series. *Advances in neural information processing systems*, *32*.

Gers, F. A., Schmidhuber, J., & Cummins, F. (2000). Learning to forget: Continual prediction with LSTM. *Neural computation*, *12*(10), 2451-2471.

Hills, J., Lines, J., Baranauskas, E., Mapp, J., & Bagnall, A. (2014). Classification of time series by shapelet transformation. *Data mining and knowledge discovery*, *28*(4), 851-881.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, *9*(8), 1735-1780.

Kazemi, S. M., Goel, R., Eghbali, S., Ramanan, J., Sahota, J., Thakur, S., Wu, S., Smyth, C., Poupart, P., & Brubaker, M. (2019). Time2vec: Learning a vector representation of time. *arXiv preprint arXiv:1907.05321*.

Kidger, P., Morrill, J., Foster, J., & Lyons, T. (2020). Neural controlled differential equations for irregular time series. *Advances in neural information processing systems*, *33*, 6696-6707.

Liu, X., Qiu, X., Wu, X., Li, Z., Guo, C., Hu, J., & Yang, B. (2025). Rethinking Irregular Time Series Forecasting: A Simple yet Effective Baseline. *arXiv preprint arXiv:2505.11250*.

Moreno-Pino, F., Arroyo, Á., Waldon, H., Dong, X., & Cartea, Á. (2024). Rough transformers: Lightweight and continuous time series modelling through signature patching. *Advances in Neural Information Processing Systems*, *37*, 106264-106294.

Müller, M. (2007). *Information retrieval for music and motion*. Springer.

Press, O., Smith, N. A., & Lewis, M. (2022). Train Short, Test Long: Attention with Linear Biases Enables Input Length Generalization. *International Conference on Learning Representations.*

Sakoe, H., & Chiba, S. (2003). Dynamic programming algorithm optimization for spoken word recognition. *IEEE transactions on acoustics, speech, and signal processing*, *26*(1), 43-49.

Tipirneni, S., & Reddy, C. K. (2022). Self-Supervised Transformer for Sparse and Irregularly Sampled Multivariate Clinical Time-Series. *ACM Transactions on Knowledge Discovery from Data.*

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, *30.*

Wei, Y., Peng, J., He, T., Xu, C., Zhang, J., Pan, S., & Chen, S. (2023). Compatible Transformer for Irregularly Sampled Multivariate Time Series. *arXiv preprint arXiv:2310.11022.*

Williams, C. K., & Rasmussen, C. E. (2006). *Gaussian processes for machine learning* (Vol. 2). MIT press Cambridge, MA.

Wu, H., Xu, J., Wang, J., & Long, M. (2021). Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Advances in neural information processing systems*, *34*, 22419-22430.

Ye, L., & Keogh, E. (2009). Time series shapelets: a new primitive for data mining. *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*, 947-956.

Yulia, R., Chen Ricky, T., & David, D. (2019). Latent ODEs for irregularly-sampled time series. *Advances in Neural Information Processing Systems*, *32.*

Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., & Zhang, W. (2021). Informer: Beyond efficient transformer for long sequence time-series forecasting. *Proceedings of the AAAI conference on artificial intelligence*, *35*(12), 11106-11115.

Zhou, T., Ma, Z., Wen, Q., Wang, X., Sun, L., & Jin, R. (2022). Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. *International conference on machine learning*, 27268-27286.

Apéndice A

Fundamentos de redes neuronales y optimización

A.1. Introducción

El objetivo de este anexo es presentar de manera autocontenida los conceptos básicos necesarios para comprender el funcionamiento de los modelos de redes neuronales utilizados en esta memoria, incluyendo:

- El modelo de *neurona artificial* y su interpretación geométrica.
- La construcción de redes neuronales feedforward como composiciones de capas.
- El proceso de entrenamiento basado en funciones de pérdida, descenso de gradiente y retropropagación (*backpropagation*).
- El papel de las funciones de activación y sus propiedades.
- El algoritmo de optimización Adam, utilizado en los experimentos de esta memoria.

Aunque el cuerpo principal del documento se centra en arquitecturas específicas como los Transformers para series de tiempo, todos estos modelos se apoyan en los mismos bloques básicos: capas lineales, funciones de activación, composición de funciones diferenciables y algoritmos de optimización sobre grandes espacios de parámetros.

A.2. Neurona artificial

A.2.1. Modelo básico

La unidad fundamental en una red neuronal es la **neurona artificial** o **perceptrón**, que puede verse como un modelo matemático que recibe un vector de entrada, aplica una transformación lineal y luego una función no lineal.

Sea $x \in \mathbb{R}^d$ un vector de entrada (por ejemplo, mediciones de sensores en un instante de tiempo), $w \in \mathbb{R}^d$ un vector de pesos y $b \in \mathbb{R}$ un término de sesgo (*bias*). El modelo de neurona se escribe como:

$$z = w^\top x + b, \tag{A.1}$$

$$y = \phi(z), \tag{A.2}$$

donde:

- z es la combinación lineal de la entrada.
- ϕ es la **función de activación**, que introduce no linealidad.

  Nota: Se utiliza una función de activación porque la combinación lineal de transformaciones lineales es, en sí misma, una transformación lineal; por lo tanto, la predicción estaría limitada a funciones objetivo lineales.

- y es la salida de la neurona.

A.2.2. Interpretación geométrica

Desde un punto de vista geométrico, la expresión $w^\top x + b = 0$ describe un hiperplano en $\mathbb{R}^d$. La neurona aprende un vector w y un sesgo b que determinan este hiperplano, y la función de activación ϕ determina cómo se asignan valores de salida a cada lado de dicho hiperplano.

Por ejemplo, si ϕ es una función escalón (no diferenciable, pero útil como intuición), la neurona implementa un clasificador lineal que decide a qué lado del hiperplano pertenece el punto x.

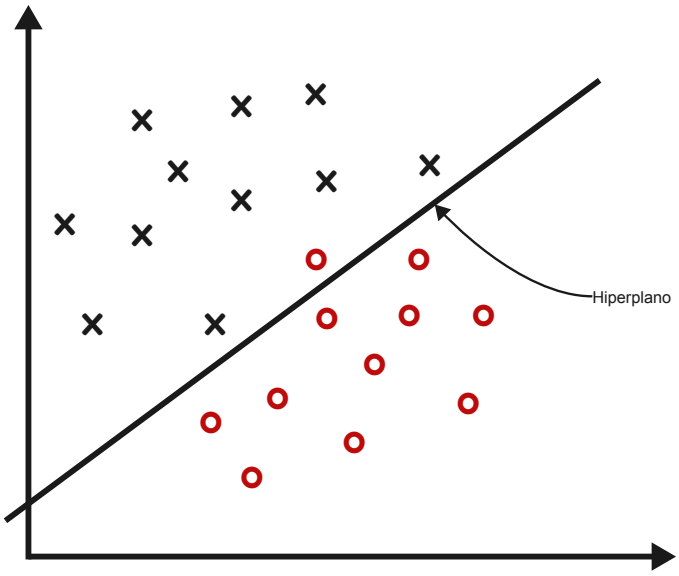

Figura A.1: Ejemplo bidimensional de un hiperplano que separa dos clases linealmente separables: las cruces negras pertenecen a una clase y los círculos rojos a la otra.

A.3. Redes neuronales feedforward

A.3.1. Capas densas y notación matricial

Una **capa densa** (o *fully-connected*) agrupa muchas neuronas en paralelo. Sea $x \in \mathbb{R}^{d_{\text{in}}}$ la entrada a la capa. Una capa densa con d_{out} neuronas se define como:

$$z = Wx + b, \quad z \in \mathbb{R}^{d_{\text{out}}}, \tag{A.3}$$

$$y = \phi(z), \tag{A.4}$$

donde:

- $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ es la matriz de pesos.
- $b \in \mathbb{R}^{d_{\text{out}}}$ es el vector de sesgo.
- ϕ se aplica componente a componente. ($\phi : \mathbb{R}^{d_{\text{out}}} \to \mathbb{R}^{d_{\text{out}}}$, donde $\phi(z) = (\phi(z_i))_i$)

Para un conjunto de N ejemplos, se suele apilar las entradas en una matriz $X \in \mathbb{R}^{N \times d_{\text{in}}}$ y la salida en $Y \in \mathbb{R}^{N \times d_{\text{out}}}$.

A.3.2. Composición de capas

Una **red neuronal feedforward** (o *Multilayer Perceptron*, MLP) consiste en la composición de varias capas de este tipo. Por ejemplo, con dos capas ocultas:

$$h_1 = \phi_1(W_1 x + b_1), \tag{A.5}$$

$$h_2 = \phi_2(W_2 h_1 + b_2), \tag{A.6}$$

$$\hat{y} = W_3 h_2 + b_3, \tag{A.7}$$

$$\hat{y}_1 \quad \hat{y}_2 \quad \hat{y}_{\text{out}}$$

donde:

- x es la entrada.
- h_1, h_2 son representaciones internas (*features* aprendidas).
- $\hat{y}$ es la salida del modelo (por ejemplo, una predicción de la serie de tiempo).

Este esquema se generaliza a un número arbitrario de capas. La capacidad de aproximación universal de las redes neuronales feedforward (bajo condiciones suaves sobre las funciones de activación) asegura que, con suficiente ancho y profundidad, pueden aproximar funciones muy complejas.

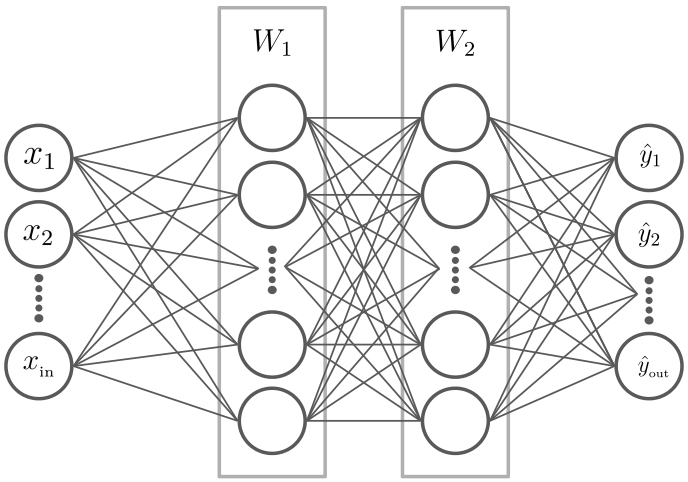

Figura A.2: Esquema de una red neuronal *feedforward* con dos capas ocultas: los vectores de entrada $x_1, \ldots, x_{\text{in}}$ se transforman mediante las matrices de pesos W_1 y W_2 en representaciones internas, para finalmente producir las salidas $\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_{\text{out}}$.

A.4. Función de pérdida

A.4.1. Pérdida para problemas de regresión

En esta memoria el problema principal es de **regresión**: se quiere predecir un valor continuo (por ejemplo, el valor de una serie de tiempo en un instante futuro). Sea $\hat{y}_i$ la predicción del modelo para el ejemplo i y y_i el valor real. La función de pérdida más habitual es el **error cuadrático medio** (*Mean Squared Error*, MSE):

$$\mathcal{L}_{\text{MSE}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \|\hat{y}_i - y_i\|^2, \tag{A.8}$$

donde θ representa el conjunto de todos los parámetros del modelo (todas las matrices W y vectores b).

El objetivo del entrenamiento es encontrar θ que minimice esta función de pérdida sobre el conjunto de datos de entrenamiento.

Otras pérdidas útiles son, por ejemplo:

- **Error absoluto medio** (MAE):

$$\mathcal{L}_{\text{MAE}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \|\hat{y}_i - y_i\|_1.$$

- **Pérdida de Huber**, que combina propiedades de MSE y MAE, siendo más robusta a outliers.

A.4.2. Regularización en la función de pérdida

Para evitar sobreajuste (*overfitting*), a menudo se agrega un término de **regularización** a la pérdida. Por ejemplo, regularización L_2 (también conocida como *weight decay*):

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{datos}}(\theta) + \lambda \sum_j \|W_j\|_F^2, \tag{A.9}$$

donde $\lambda > 0$ controla la fuerza de la regularización, y la suma recorre las matrices de pesos W_j de cada capa.

A.5. Descenso de gradiente y retropropagación

A.5.1. Descenso de gradiente

El procedimiento básico para ajustar los parámetros de la red consiste en aplicar **descenso de gradiente**. Dada una función de pérdida $\mathcal{L}(\theta)$, el descenso de gradiente estándar actualiza los parámetros según:

$$\theta_{t+1} = \theta_t - \eta \, \nabla_\theta \mathcal{L}(\theta_t), \tag{A.10}$$

donde:

- θ_t es el vector de parámetros en el paso t.
- $\eta > 0$ es la tasa de aprendizaje (*learning rate*).
- $\nabla_\theta \mathcal{L}(\theta_t)$ es el gradiente de la pérdida respecto de los parámetros.

En la práctica, los datasets suelen ser grandes, por lo que calcular el gradiente sobre todos los datos en cada paso es costoso. Por ello se utiliza **descenso de gradiente estocástico** (SGD) o variantes *mini-batch*, en las que el gradiente se estima a partir de un subconjunto de ejemplos:

$$\theta_{t+1} = \theta_t - \eta \, \nabla_\theta \mathcal{L}_{\mathcal{B}}(\theta_t), \tag{A.11}$$

donde $\mathcal{B}$ es un mini-batch de índices de entrenamiento.

A.5.2. Retropropagación (*backpropagation*)

Para poder aplicar descenso de gradiente, se requiere calcular el gradiente de la pérdida respecto de todos los parámetros de la red. Esto se logra mediante el algoritmo de **retropropagación**, que se basa en la regla de la cadena del cálculo diferencial.

A nivel conceptual, la red se puede ver como una composición de funciones:

$$\hat{y} = f_L \circ f_{L-1} \circ \cdots \circ f_1(x; \theta), \tag{A.12}$$

donde cada f_ℓ representa una capa y depende de un subconjunto de parámetros θ_ℓ.

La regla de la cadena indica que:

$$\frac{\partial \mathcal{L}}{\partial \theta_\ell} = \frac{\partial \mathcal{L}}{\partial f_\ell} \cdot \frac{\partial f_\ell}{\partial \theta_\ell}, \tag{A.13}$$

y que las derivadas $\frac{\partial \mathcal{L}}{\partial f_\ell}$ se pueden calcular recursivamente desde la salida hacia la entrada.

En términos prácticos:

1. **Paso forward:** se propaga la entrada a través de la red y se calcula la pérdida.
2. **Paso backward:** se calcula el gradiente de la pérdida respecto de la salida y se va propagando hacia atrás, capa por capa, acumulando las derivadas parciales.
3. **Actualización:** se actualizan los parámetros usando el gradiente calculado y el algoritmo de optimización elegido (SGD, Adam, etc.).

En frameworks modernos como PyTorch, este proceso se implementa automáticamente mediante *autograd*, siempre que las operaciones definidas en el modelo sean diferenciables.

A.6. Funciones de activación

A.6.1. Requisitos básicos

La función de activación ϕ cumple dos roles fundamentales:

- Introducir **no linealidad**. Sin funciones de activación, la composición de capas lineales sigue siendo una transformación lineal global.
- Ser **diferenciable** (al menos por tramos) para poder aplicar retropropagación.

A continuación se describen algunas de las funciones de activación más utilizadas.

A.6.2. Sigmoide y tangente hiperbólica

La función **sigmoide** se define como:

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \tag{A.14}$$

Sus valores están en el intervalo $(0, 1)$ y su derivada es:

$$\sigma'(z) = \sigma(z)\,(1 - \sigma(z)). \tag{A.15}$$

La **tangente hiperbólica** se define como:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \tag{A.16}$$

con valores en $(-1, 1)$ y derivada:

$$\tanh'(z) = 1 - \tanh^2(z). \tag{A.17}$$

Ambas funciones se usaron ampliamente en redes neuronales clásicas, pero presentan el problema de **gradientes que se desvanecen** (*vanishing gradients*) para valores de $|z|$ grandes, lo que dificulta el entrenamiento de redes profundas.

A.6.3. ReLU y variantes

La **Rectified Linear Unit** (ReLU) se define como:

$$\mathrm{ReLU}(z) = \text{máx}(0, z). \tag{A.18}$$

Su derivada es:

$$\mathrm{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z \leq 0. \end{cases}$$

Las ReLU han sido muy exitosas en redes profundas porque mitigan parcialmente el problema de gradientes que se desvanecen en la región $z > 0$ y son computacionalmente simples. Sin embargo, pueden producir neuronas "muertas" (salida cero constante) si los pesos se actualizan de forma que z permanezca en la región negativa.

Variantes como **Leaky ReLU** introducen un pequeño gradiente en la región negativa:

$$\mathrm{LeakyReLU}(z) = \begin{cases} z, & z > 0, \\ \alpha z, & z \leq 0, \end{cases}$$

con α pequeño (por ejemplo, $\alpha = 0{,}01$).

En modelos más recientes, incluyendo Transformers, es frecuente usar activaciones más suaves como **GELU** (*Gaussian Error Linear Unit*), que puede verse como una versión suavizada de ReLU con ciertas propiedades teóricas favorables.

A.7. Problemas en el entrenamiento de redes profundas

A.7.1. Gradientes que se desvanecen y explotan

En redes profundas, al aplicar retropropagación, se multiplican muchas matrices y derivadas de funciones de activación. Esto puede llevar a que:

- Los gradientes se hagan extremadamente pequeños (**vanishing gradients**), dificultando el aprendizaje en capas tempranas.
- Los gradientes crezcan de forma descontrolada (**exploding gradients**), causando inestabilidad numérica.

Existen varias estrategias para mitigar estos problemas:

- Diseñar buenas **inicializaciones** de los pesos (por ejemplo, He o Xavier).
- Utilizar funciones de activación adecuadas (ReLU, GELU, etc.).
- Aplicar **normalizaciones**, como *Batch Normalization* o *Layer Normalization*.
- Aplicar **clipping de gradiente**, limitando la norma del gradiente en cada actualización.

A.7.2. Regularización y generalización

Además de los términos de regularización en la pérdida, existen técnicas adicionales para mejorar la capacidad de generalización:

- **Dropout**: durante el entrenamiento, se anulan aleatoriamente algunas neuronas (se ponen a cero) con cierta probabilidad, lo que fuerza al modelo a no depender excesivamente de un subconjunto pequeño de unidades.
- **Early stopping**: se detiene el entrenamiento cuando la pérdida de validación deja de mejorar, evitando sobreajuste.
- **Data augmentation**: en dominios como imágenes o texto, se generan ejemplos adicionales modificando ligeramente los datos originales. En series de tiempo esto es más delicado, pero existen técnicas específicas.

A.8. Optimizadores avanzados: Adam

A.8.1. Motivación

El descenso de gradiente estándar utiliza una única tasa de aprendizaje η para todos los parámetros y no tiene memoria explícita del historial de gradientes. En problemas complejos y de alta dimensión, esto puede hacer que:

- El entrenamiento sea sensible a la elección de η.
- El modelo tarde en converger o quede atrapado en regiones poco favorables.

Por ello se han desarrollado **optimizadores avanzados** que adaptan la tasa de aprendizaje de manera automática, utilizando información adicional sobre la historia del gradiente. Uno de los más usados en la práctica es **Adam** (*Adaptive Moment Estimation*).

A.8.2. Algoritmo Adam

Sea θ_t el vector de parámetros en el paso t y $g_t = \nabla_\theta \mathcal{L}_t(\theta_t)$ el gradiente de la pérdida calculado en ese paso (por ejemplo, sobre un mini-batch). Adam mantiene dos variables auxiliares:

- m_t: estimación del primer momento (media) del gradiente.
- v_t: estimación del segundo momento (varianza no centrada) del gradiente.

El algoritmo puede describirse como sigue:

1. Inicializar $m_0 = 0$, $v_0 = 0$, $t = 0$.
2. En cada paso:

$$t \leftarrow t + 1, \tag{A.19}$$
$$g_t \leftarrow \nabla_\theta \mathcal{L}_t(\theta_t), \tag{A.20}$$
$$m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t, \tag{A.21}$$
$$v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \tag{A.22}$$

donde las operaciones son componente a componente y $0 < \beta_1, \beta_2 < 1$ son hiperparámetros (típicamente $\beta_1 \approx 0{,}9$, $\beta_2 \approx 0{,}999$).

3. Para corregir el sesgo inicial (ya que m_t y v_t parten en cero), se calculan:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \tag{A.23}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}. \tag{A.24}$$

4. Finalmente, se actualizan los parámetros:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}, \tag{A.25}$$

   donde η es la tasa de aprendizaje base y ε es un pequeño valor positivo (por ejemplo, 10^{-8}) para evitar divisiones por cero.

En palabras simples:

- m_t actúa como un **promedio móvil** del gradiente (similar a *momentum*).
- v_t actúa como un promedio móvil de la magnitud cuadrática del gradiente, permitiendo **adaptar** la escala de los pasos por parámetro.
- La división por $\sqrt{\hat{v}_t}$ hace que los parámetros con gradientes consistentemente grandes tengan pasos de actualización más pequeños, y viceversa.

En esta memoria se utiliza Adam como optimizador por defecto para entrenar el modelo Transformer, tal como se especifica en los archivos de configuración de entrenamiento.

A.9. Relación con el modelo Transformer de la memoria

Aunque el Transformer parece, a primera vista, una arquitectura muy distinta a un MLP estándar, en el fondo reutiliza los mismos componentes básicos descritos en este anexo:

- Cada bloque de **atención multi-cabeza** está compuesto por proyecciones lineales (capas densas) que calculan las matrices de *queries*, *keys* y *values*, seguidas de operaciones de atención y una proyección lineal de salida.
- El **bloque feed-forward** dentro de cada capa del Transformer es esencialmente un MLP de dos capas aplicado posición a posición, con una función de activación no lineal (por ejemplo, ReLU o GELU).
- La red completa se entrena minimizando una función de pérdida (en este caso, típicamente MSE sobre las predicciones de la serie de tiempo) usando descenso de gradiente y un optimizador como Adam.
- La **cabeza de regresión** sobre el token *target* en el modelo de esta memoria es, nuevamente, una capa lineal (o una pequeña MLP) que transforma la representación latente del token en la predicción final.

Por lo tanto, comprender el funcionamiento de neuronas, funciones de activación, retropropagación y Adam es suficiente para entender el entrenamiento de toda la arquitectura propuesta, aunque los detalles de la atención y los encodings temporales se desarrollen en capítulos específicos del cuerpo principal.

A.10. Resumen del anexo

En este anexo se han presentado los fundamentos necesarios para comprender el entrenamiento de redes neuronales modernas:

- El modelo de neurona artificial como combinación lineal más función de activación.
- La construcción de redes feedforward mediante la composición de capas densas.
- La definición de funciones de pérdida para problemas de regresión y el uso de regularización.
- El algoritmo de entrenamiento basado en descenso de gradiente, estimado mediante mini-batches, y el cálculo eficiente de gradientes mediante retropropagación.
- El rol de las funciones de activación y los problemas asociados al entrenamiento de redes profundas (gradientes que se desvanecen o explotan).
- El algoritmo Adam como optimizador adaptativo ampliamente utilizado en la práctica.

Estos conceptos constituyen la base sobre la cual se construyen las arquitecturas específicas analizadas en la memoria, en particular el modelo Transformer adaptado a series de tiempo no equiespaciadas.